

TEAM 05 · HiBee

2차 보고서

돌핀팟 (DolphinPod)

스마트폰 사용 패턴을 분석하여 게임형 보상 시스템으로
사용 습관 개선을 돕는 어플리케이션

1. Team Info (팀 정보)

1.1 과제명

스마트폰 사용 패턴을 분석하여 게임형 보상 시스템으로 사용 습관 개선을 돕는 어플리케이션 — 돌핀팟 (DolphinPod)

1.2 팀 정보

항목	내용
팀 번호	TEAM 05
팀 이름	HiBee

1.3 팀 구성원

이름	역할	담당 업무
김호연	AI	AI 파이프라인 설계 및 구현, 멀티 에이전트 기반 품질 검증, AI 및 시스템 통합 테스트
이승현	BE / DB	BE 및 DB 설계, DB 구조 및 RAG 설계, API 설계 및 구현, DB와 API 구조 고도화 및 성능 검증
양시은	FE / UI	사용자 흐름·화면·멀티 에이전트 설계, API 연동 및 사용자 입력 상태 처리, UI 안정화·발표 자료 디자인 및 시각화

2. Project-Summary (과제 요약)

2.1 문제 정의

과학기술정보통신부의 스마트폰 과의존 실태조사(2023)에 따르면, 국내 스마트폰 이용자 10명 중 8명이 자신의 스마트폰 과의존 위험성을 인지하고 있다. 과학기술정보통신부의 스마트폰 과의존 실태조사(2025)에 따르면, 스마트폰 과의존 위험군 10명 중 8명은 본인이 스마트폰을 과다 이용한다는 것을 인지하고 있다.¹ 그러나 인식과 실제 행동 변화 사이에는 큰 간극이 존재하며, 그 핵심 원인은 세 가지로 정리된다.

① 사용 패턴 이해 부족

자신의 스마트폰 사용 데이터 패턴을 체계적으로 인지하지 못하여, '하루에 몇 시간 썼다'는 수치 외에 언제·어떤 맥락에서·어떤 방식으로 의존하게 되는지에 대한 의미 있는 인사이트나 개선 방향을 얻기 어렵다.

② 습관 지속 동기 부족

기록이나 관리를 통해 개선하고 싶지만, 지속적인 동기 및 재미 요소가 부족해 긍정적인 습관 형성에 이르지 못한다.

③ 사용자 자율성이 떨어지는 차단 방식

기존 서비스들의 강제적인 차단 방식에 대한 거부감으로 습관을 고치려는 시도 자체를 포기하게 된다. 차단이 해제된 후에도 동일한 패턴이 반복되는 근본적인 한계가 있다.

이러한 문제점을 해결하기 위해 돌핀 팟(Dolphin Pod)은 자신의 디지털 행동 패턴을 이해하고 스스로 더 나은 사용 방식을 만들어 가고 싶은 사용자를 위한 서비스를 기획하기 시작했다.

Pain Point — Solution 매핑

Pain Point	Solution
사용 패턴 이해 부족	① 데일리 체크인 기반 사용 패턴 인식
	② 사용 데이터 + 개인화 정보 기반 AI 리포트 제공

¹ 과학기술정보통신부, 『2025년 스마트폰 과의존 실태조사』,

<https://www.msit.go.kr/publicinfo/view.do?sCode=user&mId=63&mPid=62&publictSeqNo=443&publictListSeqNo=12&formMode=R&referKey=443%2C12>

Pain Point	Solution
습관 지속 동기 부족	③ 소셜 기능을 통한 지속 동기 강화
사용자 자율성이 떨어지는 차단 방식	④ 게이미피케이션 기반 자극 대체, 자기 인식

목표 사용자

돌핀팟의 타겟 사용자는 자신의 디지털 행동 패턴을 이해하고, 스스로 더 나은 사용 방식을 만들어가고 싶은 사용자이다. 구체적으로는 다음과 같은 특성을 가진 사용자를 주요 타겟으로 설정한다.

- 스마트폰 사용 기록 데이터를 유의미하게 기록하고 싶은 사용자
- 자신의 행동 패턴을 데이터로 이해하고 싶은 사용자
- 강제적인 방식보다는 스스로 개선하고 싶은 사용자
- 소셜 기능을 통한 지속적인 동기 부여를 선호하는 사용자
- 자기관리와 습관 개선에 관심이 있는 사용자

돌핀팟의 핵심 타겟은 자신의 스마트폰 과다 사용을 인지하고 있으나, 강제 차단형 서비스의 통제에 거부감을 느껴 자율적 개선 방법을 찾는 사용자이다. 구체적인 목표 사용자는 다음과 같이 단계적으로 설정한다.

1차 타겟 — 자기주도형 디지털 웰빙 추구 20대 대학생, 사회초년생

- 인구통계: 만 19~29세, Android 스마트폰 주 사용자, 모바일 앱 활용에 능숙
- 행동 특성:
 - 일평균 스마트폰 사용 시간 4시간 이상, 인스타그램과 유튜브 등 숏폼 콘텐츠 소비 경향 높음
 - 강제 차단 앱(Forest 등)으로 스마트폰 사용 시간을 줄이려고 시도하였다가, 강제 차단에 대한 거부감으로 이탈한 경험이 있음
 - 가계부·운동 기록 등 데이터 기반 자기관리 도구에 친숙함
 - 인스타그램·러닝 크루 등 소셜 커뮤니티 기반 활동을 통한 동기 부여에 익숙함
- 동기 특성:
 - 외재적 통제보다 자기 데이터에 기반한 인사이트와 자율적 행동 변화를 선호
 - 자기 인식과 회고를 통한 점진적 개선에 가치를 둠

2차 타겟 — 자기계발 지향 20~30대 직장인

시간 관리, 집중력 회복, 워라밸 개선을 위해 디지털 사용 습관을 점검하고자 하는 사용자. 1차 타겟과 동기 구조는 유사하나, 사용 맥락이 학업 생산성에서 업무와 일상의 생산성으로 이동한다.

2.2 기존 서비스와의 비교

현재 시장에 출시된 주요 스마트폰 사용 관리 서비스들은 각각 일부 기능만을 제공하며 다음과 같은 한계를 가진다.

서비스	제공 기능	한계
Forest	앱 강제 차단 / 집중 타이머	사용자 자율성 부족, 차단 해제 후 동일 패턴 반복
Rize	사용 시간 집계	단순 수치만 제공, 패턴 분석 및 개선 방향성 없음
Focus Friend	게임화 요소 (혼자)	혼자 해야 하므로 지속할 동기 부족
MindDoc / Wysa / Woebot	대화형 상담 / 감정 추적 / 리포트	시각적 몰입 요소 부재, 사용 데이터 자동 수집 없음
Quabble	멘탈 관리 / 감정 기록	사용자가 데이터를 수동으로 입력해야 함

(1) Forest

Forest는 사용자가 설정한 시간 동안 스마트폰을 사용하지 않으면 가상의 나무가 자라고, 중도에 앱을 닫으면 나무가 죽는 구조의 강제 차단형 집중 앱이다. 명확한 시각적 보상과 단순한 작동 방식으로 다수의 사용자를 확보했으나, **강제적인 스마트폰 차단에 의존하는 구조적 한계**를 지닌다. 사용자가 '왜 스마트폰을 쓰는가'에 대한 자기 이해 과정 없이 단순히 차단되기 때문에, 타이머가 종료되거나 차단을 해제하는 순간 동일한 사용 패턴이 즉시 반복된다. 또한 사용 자체를 '실패'로 규정하는 이분법적 접근은 단기적으로는 효과적이지만, 장기적으로는 사용자의 자기 효능감과 자율적 동기를 약화시킨다.

(2) Rize

Rize는 앱·웹사이트 사용 시간을 카테고리별로 집계하여 시각화하는 트래킹 도구이다. 일·주·월 단위 사용 통계를 객관적 수치로 제시한다는 점에서 자기 인식의 출발점은 마련하지만, 수치의 나열에 그치는 한계가 있다. 사용자가 '어제 SNS에 4시간을 사용했다'는 정보를 받더라도, **그 사용이 어떤 시간대·맥락에서 집중되었는지, 어떤 트리거가 반복되는지, 무엇을 어떻게 개선해야 하는지에 대한 해석이 제공되지 않는다.** 결과적으로 데이터는 풍부하지만 인지적 통찰로 이어지는 연결고리가 부재하여 행동 변화로 전환되지 않는다.

(3) Focus Friend

Focus Friend는 캐릭터 육성형 게이미피케이션을 도입한 집중 보조 앱이다. 시각적 몰입도와 정서적 친근감은 높지만, **사용자 1인 단독 진행 구조로 설계되어 있어 초기 흥미가 사라진 후 사용을 지속시킬 외부 동기가 부족하다.**

(4) MindDoc / Wysa / Woebot

세 서비스는 모두 **대화형 챗봇 기반의 정신건강 앱**으로, CBT(인지행동치료) 원리를 적용한 상담, 감정 추적, 주간 리포트 등을 제공한다. 임상적으로 검증된 심리치료 프레임워크를 일상 앱에 도입했다는 점에서 의의가 있으나, 두 가지 한계를 보인다. 첫째, **텍스트 위주의 인터페이스로 시각적·정서적 몰입을 유발하는 요소가 부족**하여 일상적 접근성이 떨어진다. 둘째, **사용자의 실제 스마트폰 사용 데이터를 자동으로 수집하지 않고 자기 보고(self-report)에만 의존**하므로, 객관적 행동 패턴과 주관적 인식 간의 정합성을 검증할 수 없다. 즉 사용자가 자신의 사용 실태를 정확히 인지하지 못한 상태에서 상담이 진행되어 개입의 정확도가 낮아진다.

(5) Quabble

Quabble은 멘탈 관리와 감정 기록을 캐릭터 인터페이스로 풀어낸 앱이다. 시각적 친근감과 정서적 안정감을 주는 디자인은 강점이나, **모든 데이터를 사용자가 직접 입력해야 하는 수동 입력 구조라는 점에서 한계**를 가진다. 이는 기록의 누락, 자기 보고 편향, 입력 피로로 인한 이탈을 유발하여, 장기적인 데이터 신뢰성과 사용 지속성을 떨어뜨린다.

돌핀팻의 차별점

차별점	내용
높은 사용자 자율성	사용 패턴 분석 후 사용자 스스로 KPT 회고·개선 → 자율적인 패턴 선택 유도
AI 기반 사용 패턴 세분화	단순 사용 시간 집계를 넘어, AI 기반 패턴 분석 및 특이 행동 탐지 제공
지속적인 동기 부여	게이미피케이션으로 동기를 유지하고, 그룹 챌린지로 장기적인 참여를 강화

(1) 높은 사용자 자율성 — Forest의 강제 차단 한계 보완

돌핀팻은 차단이라는 외부 통제 대신 **자기 인식 기반의 행동 변화**를 핵심 원리로 삼는다. 시스템은 사용자의 스마트폰 사용 패턴을 분석하여 제시할 뿐, 어떤 앱을 언제 쓸지에 대한 의사결정은 전적으로 사용자에게 위임한다. 특히 KPT 회고 구조(Keep-유지할 행동 / Problem-문제가 된 행동 / Try-시도해볼 개선)를 도입하여 사용자가 자신의 사용 패턴을 스스로 평가하고 개선 방향을 설정하도록 유도한다. 이는 인지행동치료(CBT)의 핵심 원리인 인지 재구성과 자기 모니터링을 일상 앱에 적용한 설계로, 강제 차단이 야기하는 '해제 후 동일 패턴 반복' 문제를 근본적으로 해결한다. 사용자가 직접 선택한 변화이기 때문에 차단 기반 앱에서 흔히 나타나는 자기 효능감 저하 없이 장기적 습관 형성으로 이어질 수 있다.

(2) AI 기반 사용 패턴 세분화 — Rize, MindDoc의 패턴 해석 부재 보완

돌핀팻은 단순한 사용 시간 집계를 넘어, **사용 데이터를 자동 수집**한 뒤 AI가 의미 있는 패턴을 추출하여 해석을 제공한다. Android의 UsageStatsManager API를 활용하여 사용자의 별도 입력 없이 앱별·시간대별 사용 데이터를 수집하므로, 자기 보고에 의존하는 기존 멘탈케어 앱의 인식-행동 격차 문제를 해소한다. 수집된 데이터는 RAG(Retrieval-Augmented Generation) 기반 AI 파이프라인을 통해 분석되어, '주말 자정 이후 SNS 사용 급증', '특정 알림 직후 평균 23분의 비계획적 사용 발생'과 같은 특이 행동 패턴을 자동으로 탐지한다. 또한 분석 결과의 신뢰성을 확보하기 위해 Writer → Deterministic Check → LLM Judge로 이어지는 3단계 QA 파이프라인을 구축하여, AI 출력의 비결정성을 최소화하였다. 이는 단순 수치 제공에 머무르는 트래킹 앱과, 사용자 입력에만 의존하는 챗봇형 앱 양쪽의 한계를 동시에 보완한다.

(3) 지속적인 동기 부여 — Focus Friend, Quabble의 동기·지속성 한계 보완

돌핀팻은 게이미피케이션과 사회적 동기 요소를 결합하여 장기적 참여를 유도한다. 돌고래 캐릭터의 성장과 '팻(pod, 무리)' 메타포를 시각적 몰입의 축으로 삼되, 1인 진행에 그치지 않고 추가적인 **그룹 챌린지** 시스템을 통해 또래 비교와 사회적 강화 효과를 제공한다. 함께 목표를 설정하고 진행 상황을 공유하는 구조는 자기 통제력이 낮은 사용자에게도 외부 동기를 안정적으로 공급하여, 단독 게이미피케이션 앱에서 흔히 발생하는 초기 흥미 소진 후 이탈 문제를 완화한다. 동시에 자동 데이터 수집을 기반으로 하기 때문에, 수동 입력형 앱에서 발생하는 입력 피로와 데이터 누락 문제도 발생하지 않는다.

2.3 제안 내용

돌핀팟(DolphinPod)은 Android UsageStatsManager API로 수집한 **스마트폰 사용 데이터를 세션 단위로 구조화**하고, **AI가 특이 사용 패턴 5가지 후보를 자연어 문장으로 생성**해 사용자에게 제시하는 AI 기반 Android 어플리케이션이다. 단순히 사용 시간을 수치로 보여주는 데 그치지 않고, "늦은 밤 SNS를 반복적으로 확인했어요"와 같이 **사용자가 자신의 행동을 직관적으로 인식할 수 있는 형태로 데이터를 전달**하는 것을 핵심으로 한다.

사용자는 AI가 제시한 패턴 후보 중 기록으로 남기고 싶은 항목을 선택하고, KPT(Keep/Problem/Try) 프레임워크에 따라 이어가고 싶은 패턴(K), 고치고 싶은 패턴(P), 개선하고 싶은 패턴(T)으로 분류하여 회고를 작성한다. 이렇게 축적된 KPT 데이터와 온보딩 시 수집한 초기 정보를 결합하여 AI가 데일리·주간 개인화 리포트를 생성하며, **사용자는 단순 데이터 열람을 넘어 자신의 행동 패턴에 대한 구체적인 인사이트를 얻을 수 있다.**

자기인식만으로는 습관 변화가 오래 지속되기 어렵다는 점을 고려하여, **게이미피케이션과 소셜 기능**으로 지속적인 동기 부여 구조를 마련한다. 체크인 및 그룹 챌린지 달성 시 코인을 획득하고 이를 통해 개인 캐릭터를 커스터م할 수 있으며, 친구와 함께하는 그룹 챌린지와 응원 기능은 혼자가 아닌 커뮤니티 안에서 변화를 만들어가는 경험을 제공한다. 핵심 철학은 "차단이 아닌 자기인식"이다.

2.4 기대 효과 및 의의

- 사용자 측면: 스마트폰 사용 데이터 분석을 통해 강제적인 차단이 아닌 자율적 사용 습관 개선을 지원한다. 그리고, KPT 회고 프레임워크와 AI 개인화 리포트를 결합하여 지속적인 자기 모니터링 습관을 형성할 수 있다.
- 사회적 측면: 디지털 웰빙 앱 시장에서 차별화된 자기인식 기반 서비스로 출시하여 학교 미디어 리터러시 교육 도구 및 청소년 과의존 예방 사업과 연계하여 사회적 가치 창출을 기대할 수 있다.
- 확장 가능성: 수면·운동·식습관 등 타 생활습관 영역으로 AI 개인화 행동 변화 플랫폼 확장 가능

2.5 주요 기능 리스트

돌핀팟의 주요 기능은 앞서 정의한 세 가지 페인포인트(① 사용 패턴 이해 부족 ② 습관 지속 동기 부족 ③ 사용자 자율성이 떨어지는 차단 방식)를 단계적으로 해결하는 메커니즘으로 설계되었다. **데일리 체크인과 AI 개인화 리포트**는 자동 수집된 데이터에 의미 해석을 부여하여 PP①을 해소하고, **소셜 기능**은 외부 동기 공급을 통해 PP②를 보완하며, **게이미피케이션**은 강제 차단 방식에 대체되는 자극을 제공하는 구조로 PP③에 대응한다.

기능	설명
1. 데일리 체크인	매일 지정 시간 푸시 알림 → 당일 사용 데이터 세션 전처리 → AI가 특이 패턴 5개 후보 제시 → 사용자 KPT 선택·기록
2-1. AI 개인화 리포트 (데일리)	KPT 패턴 / 사용 흐름 / 주 사용 카테고리 + AI 코멘트 생성
2-2. AI 개인화 리포트 (주간)	데일리 리포트 경향 분석 + 점수·추세·체크인 참여 횟수 + AI 코멘트
2-3. 캘린더 조회	데일리/주간 리포트를 캘린더에서 날짜별 조회
3. 소셜 기능	친구 커넥션 / 응원 알림 / 일주일 단위 그룹 챌린지
4. 게이미피케이션	업적 배지 / 코인 보상 / 캐릭터·배경·엑세서리 커스텀
5. 메인화면 & 대시보드	사용 데이터 시각화

(1) 데일리 체크인 — 셀프 KPT 회고: 자기 인식의 시작이 되는 핵심 기능

매일 지정 시간(기본 오후 10시) 푸시 알림을 통해 사용자에게 당일의 스마트폰 사용에 대한 짧은 회고를 유도한다. UsageStatsManager API로 자동 수집된 사용 데이터는 세션 단위로 전처리되며, AI가 그중 의미 있는 5가지 특이 패턴(예: "오후 11시 이후 SNS를 30분 이상 연속 사용")을 자연어 문장으로 제시한다. 사용자는 그 중 기록할 항목을 선택해 **KPT(Keep/Problem/Try) 프레임워크**에 따라 분류·기록한다.

이 기능은 두 가지 페인포인트를 동시에 해결한다. 자동 데이터 수집 + AI 패턴 해석으로 "사용 패턴 이해 부족"을 해소하고, 사용자가 직접 K/P/T를 분류·기록하도록 하여 외부의 강제 통제 없이 자율적 자기 인식 과정을 끌어낸다. 자기 보고에만 의존하는 멘탈 헬스케어 앱(MindDoc, Quabble)과 달리 객관적 행동 데이터에 기반하며, 강제 차단 중심 앱(Forest)과 달리 사용자가 변화의 주체가 된다.

(2-1) AI 개인화 리포트 (데일리) — AI 기반 사용 패턴 세분화

데일리 체크인을 마치면 즉시 개인화 리포트가 생성된다. 리포트는 ㉠ 당일 KPT 회고에서 추출된 핵심 패턴, ㉡ 시간대별 앱 사용 흐름 시각화, ㉢ 가장 많이 사용한 카테고리 분석, ㉣ 사용자의 KPT 기록과 사용 데이터를 함께 해석한 AI 코멘트로 구성된다. 단순 수치 나열에 그치는 트래킹 도구(Rize 등)와 달리, 데이터에 맥락적

해석을 부여한 자연어 인사이트를 제공하여 사용자가 자신의 행동 패턴을 즉각적으로 이해할 수 있도록 돕는다. 본 기능을 통해 "데이터는 있지만 해석이 없다"는 기존 트래킹 앱의 구조적 한계를 보완한다.

(2-2) AI 개인화 리포트 (주간) — 자기 인식의 일회성 한계 보완

한 주간 누적된 데일리 리포트를 종합하여 주 단위 추세를 제공한다. 일별 점수의 변동 추이, 체크인 참여 횟수, 반복적으로 등장한 K/P/T 패턴, 한 주의 행동 변화에 대한 AI 종합 코멘트가 포함된다. 데일리 리포트가 일일 단위의 자기 인식이라면, 주간 리포트는 이러한 인식을 연결해주므로 사용자가 자신의 변화 흐름을 거시적으로 인식할 수 있게 한다. 단발성 회고에 그치지 않고 **점진적 행동 변화**로 이어지도록 하는 장기적 자기 인식 장치이다.

(2-3) 캘린더 조회 — 회고의 연속성 시각화

사용자가 작성한 데일리·주간 리포트를 캘린더 인터페이스에서 날짜별로 탐색할 수 있는 기능이다. 시험 기간, 휴가, 프로젝트 마감 등 특정 시기의 사용 변화나 장기적 개선 추이를 캘린더 상에서 시각적으로 확인할 수 있어, 자기 인식의 일회성·휘발성을 극복한다. 별도의 새 데이터를 만들지 않고 기존 회고 기록의 접근성을 높이는 보조 기능이지만, 사용자가 자신의 변화를 한눈에 추적할 수 있다는 점에서 지속 사용에 도움이 된다.

(3) 소셜 기능 — 소셜 커뮤니티로 지속적인 동기 부여

사용자는 친구를 추가하여 서로의 체크인 진행 상황을 공유하고 응원 알림을 주고받을 수 있으며, **일주일 단위 그룹 챌린지**에 함께 참여할 수 있다. 자기 통제력에만 의존할 경우 동기가 빠르게 소진되는 문제를 보완하기 위해, 타인과 함께하는 기능을 통해 외부 동기를 지속적으로 부여한다. 그룹 챌린지는 공동 목표라는 책임감을 부여하여 단독 사용자가 쉽게 도달하기 어려운 **장기 지속성**을 확보하는 핵심 장치이다.

(4) 게이미피케이션 — 보상으로 대체 자극 제공

체크인 완료, 챌린지 달성 등 사용자의 행동에 따라 업적 뱃지와 코인을 획득하며, 코인은 **캐릭터·배경·액세서리 커스터마이징**에 사용된다. 강제 차단이 사용자의 자율성을 박탈하고 부정적 통제 경험을 누적시키는 반면, 게이미피케이션은 긍정적 보상 구조를 통해 사용자가 스스로 사용 습관을 개선하도록 자발적 참여를 유도한다. 또한 캐릭터의 성장과 커스터마이징이라는 시각적 몰입 요소는 회고와 챌린지를 일회성 과업이 아닌 **지속적 경험**으로 전환시킨다.

(5) 메인화면 & 대시보드 — 사용 데이터 시각화

앱 진입 시 가장 먼저 보게 되는 메인 화면에서는 **그날의 사용 데이터를 고래의 크기와 색으로 직관적으로 시각화**한 화면이다. 대시보드에서는 UsageStatsManager API로 수집된 사용 데이터를 **시간대별 그래프·카테고리별 비중·주간 추이 차트 등으로 시각화**하여, 수치 나열에 그치는 OS 기본 스크린타임 도구와 달리 사용자가 자신의 행동 패턴을 직관적으로 인식할 수 있도록 한다.

3. Project-Design (과제 설계)

3.1 요구사항 정의

기능 요구사항

돌핀팟의 기능 요구사항은 **데이터 파이프라인** 관점에서 다음과 같이 구성된다. 객관적 사용 데이터를 자동 수집하여 의미 있는 단위로 정제한 뒤, 이를 입력으로 AI가 자기 인식을 유도하는 콘텐츠를 생성한다. 동시에 사용자의 지속 참여를 위한 소셜과 게이미피케이션 모듈이 각각 병렬적으로 작동하며, 모든 AI 출력은 별도의 품질 검증 계층을 통과해야 사용자에게 전달된다.

구분	요구사항
사용 데이터 수집	Android UsageStatsManager API를 통해 앱별 사용 시간, 실행 횟수, 잠금 해제 횟수, 최장 연속 사용 시간 수집
데이터 전처리	수집된 raw 로그를 세션 단위로 구조화하여 시간대·빈도·지속시간 형태의 Daily Snapshot 생성
체크인	매일 푸시 알림 발송, AI가 특이 패턴 5개 후보 자연어 생성, 사용자 KPT 분류 및 저장
AI 리포트	데일리/주간 개인화 리포트 생성 (RAG 기반 근거 있는 피드백 포함)
소셜	친구 추가·삭제·신청, 응원 알림, 그룹 챌린지 생성·참여·조회
게이미피케이션	코인 보상 지급, 캐릭터 커스텀, 업적 배지 부여
품질 검증	체크인/리포트 생성 결과에 대한 3단계 QA : (Writer 작성 -> Deterministic 검증 -> LLM Judge)

(1) 사용 데이터 수집

Android **UsageStatsManager API**를 활용하여 1차적으로 사용자가 별도로 입력하지 않아도 정량화된 사용 데이터를 자동 수집한다. 수집 항목은 앱별 사용 시간, 실행 횟수, 잠금 해제 횟수, 최장 연속 사용 시간으로 구성되며, 단순 사용 총량을 넘어 **사용 강도와 충동성**을 측정할 수 있는 지표를 포함하게 된다. 백그라운드 수집은 **WorkManager**로 스케줄링하여 배터리 안전성을 확보하며, 자기 보고에만 의존하는 기존 멘탈케어 앱의 한계(인식-행동 격차 검증 불가)를 본 단계에서 구조적으로 해소한다.

(2) 데이터 전처리

raw 사용 로그는 그 자체로는 의미 있는 인사이트를 추출하기 어려우므로, 2차적으로 **세션 단위로** 구조화하는 전처리 단계를 거친다.. 일정 시간 이상의 비사용 구간을 기준으로 연속 사용을 하나의 세션으로 묶고,

시간대·앱·카테고리별 통계를 결합하여 **Daily Snapshot**을 생성한다. 이는 AI 모델이 의미 있는 패턴을 추출할 수 있는 표준화된 입력 형태이자, 이후 단계 전체의 데이터 기반이 된다. 수집한 데이터를 의미 있는 정보로 전환하는 전환점이 되는 단계이다..

(3) 체크인

매일 지정 시간에 푸시 알림을 발송하여 사용자를 체크인 화면으로 유도하고, 당일 Daily Snapshot을 입력으로 AI가 **특이 사용 패턴 5개 후보를 자연어 문장**으로 생성한다(Writer 모델: GPT-4o-mini, 상대적으로 토큰이 적게 필요하므로 경량 모델을 사용함). 사용자는 그중 회고하고자 하는 항목을 선택하여 KPT(Keep/Problem/Try) 프레임워크로 스스로 평가한 후 그 내용을 저장한다. 본 요구사항은 **시스템에서 받아온 데이터에 사용자가 부여한 주관적 의미**가 결합되어 자기 인식을 돕고 핵심적인 페인포인트를 해소하는 기능이다.

(4) AI 리포트

누적된 KPT 기록, Daily Snapshot, 온보딩 시 수집한 사용자 컨텍스트를 결합하여 **데일리/주간 개인화 리포트**를 생성한다. 단순 데이터 요약이 아닌 **RAG(Retrieval-Augmented Generation) 기반의 근거 있는 피드백**을 제공하는 것이 핵심이다. 구체적으로, 디지털 웰니스·CBT·수면·집중 관련 전문 지식이 적재된 “expert_knowledge” 테이블을 **Supabase + pgvector** 환경에서 **하이브리드 RRF(BM25 + 벡터 검색)** 방식으로 조회한 뒤, 검색된 문서를 컨텍스트에 주입하여 Writer 모델(Claude Sonnet, 정교한 리포트 작성을 위해 자연어 생성에 적합한 모델을 사용함)이 응답을 생성한다. 이를 통해 AI 코멘트가 전문 지식에 기반하여 구체적이고 의미있는 조언으로 작동하도록 한다.

(5) 소셜

친구 추가·삭제·신청, 응원 알림, 그룹 챌린지의 생성·참여·조회 기능을 제공한다. 그룹 챌린지는 일주일 단위로 진행되며, 참여자의 체크인·달성 진행 상황이 실시간으로 동기화된다. 응원 알림은 친구의 행동(체크인 완료, 챌린지 달성 등) 트리거에 대응하여 푸시 알림 형태로 전송된다. 본 요구사항은 ‘지속할 동기 부족’이라는 페인포인트 해결을 위한 설계로, **사용자 간 사회적 책임감**을 형성하여 단독 사용 대비 장기 지속률을 확보하는 것을 목적으로 한다.

(6) 게이미피케이션

체크인 완료, 챌린지 달성 등 사용자의 행동 이벤트에 대해 코인 보상을 지급하고, 누적 코인은 **캐릭터·배경·액세서리 커스터마이징**에 사용된다. 또한 특정 행동 조건(예: 7일 연속 체크인) 충족 시 업적 배지가 자동 부여된다. 강제 차단 of 부정적 통제 경험을 **긍정적 보상 구조**로 대체함으로써 ‘강제 차단 거부감’ 페인포인트 해결의 대체 보상을 제공한다.

(7) 품질 검증

본 시스템은 LLM 기반 콘텐츠 생성에 의존하므로 **자연어 출력에 대한 검증**이 큰 기술적 과제이다. 이를 해결하기 위해 모든 AI 출력(체크인 패턴 후보, 데일리·주간 리포트)은 사용자에게 노출되기 전에 **3단계 품질 검증**을 거친다.

- **Writer 작성**: 할루시네이션을 방지하기 위한 실제 데이터 기반 답변 + 근거 없는 답변 금지 프롬프트를 기반으로 1차적으로 코멘트를 생성한다.
- **Deterministic 검증**: 규칙 기반 검증으로 형식·길이·금칙어(임상 용어 누설 방지)·시간 표기 규칙(59초 이하 생략, 60초 이상 분 단위 올림) 등을 자동 점검한다.
- **LLM Judge**: 정성적 기준(어조의 적절성, 인사이트의 구체성, 사용자 친화성)을 별도 LLM이 평가한다.

검증 실패 시 최대 3회 재시도하며, 모든 시도에서 통과하지 못한 경우 **사전 정의된 폴백 응답**을 노출한다. 이 파이프라인은 사용자에게 일관된 품질의 경험을 제공함과 동시에, AI 시스템의 신뢰성을 정량적으로 관리할 수 있게 하는 본 프로젝트의 핵심 기술적 차별점이다.

비기능 요구사항

비기능 요구사항은 본 시스템의 운영 환경, 사용자 접근성, AI 시스템 특유의 신뢰성 확보를 위한 기술적 인프라를 정의한다. 각 항목은 단순한 기술 선택의 결과가 아니라, 앞서 정의한 기능 요구사항이 안정적으로 작동하기 위한 전제 조건으로 도출되었다.

구분	요구사항
플랫폼	Android
인증	Google OAuth 기반 로그인
배포	Google Play Store 등록
CI/CD	GitHub Actions + PromptFoo CI를 통한 AI 프롬프트 회귀 테스트 자동화

(1) 플랫폼 — Android

본 서비스는 Android 플랫폼을 단독 지원 대상으로 한다. 핵심 데이터 수집 기능이 Android의 **UsageStatsManager API**를 기반으로 하기 때문이다. 조사 결과, iOS의 Screen Time API는 앱별 상세 사용 데이터에 대한 외부 접근을 제한하고 있어 동일한 수준의 자동 데이터 수집이 구조적으로 불가능하였다. iOS 확장은 별도의 데이터 수집 전략 수립 이후 확장 항목으로 검토한다.

(2) 인증 — Google OAuth

사용자 인증은 자체 ID·비밀번호 시스템이 아닌 **Google OAuth 2.0 기반 SSO**로 통일한다. Android 사용자 대부분이 이미 Google 계정을 보유하고 있어 회원가입 단계의 마찰을 최소화하며, 비밀번호 관리·재설정·해싱 등 자체 인증 시스템 운영에 따르는 보안 부담을 줄이기 위해서이다. 인증 범위는 사용자 식별을 위한 최소 정보로 한정하여 개인정보 노출을 최소화한다.

(3) 배포 — Google Play Store

배포 채널은 **Google Play Store**로 단일화한다. 가장 큰 Android 앱 마켓플레이스로서 잠재 사용자에게 대한 도달 범위가 넓고, Google이 제공하는 앱 심사 절차를 통해 보안·기능 안정성에 대한 1차적 신뢰가 확보된다. 또한 자동 업데이트, 버전 관리, 사용자 리뷰 수집 등 운영에 필요한 기본 인프라가 내장되어 있어, 자체 배포 시스템 구축 없이 표준화된 출시·운영 프로세스를 빠르게 확립할 수 있다.

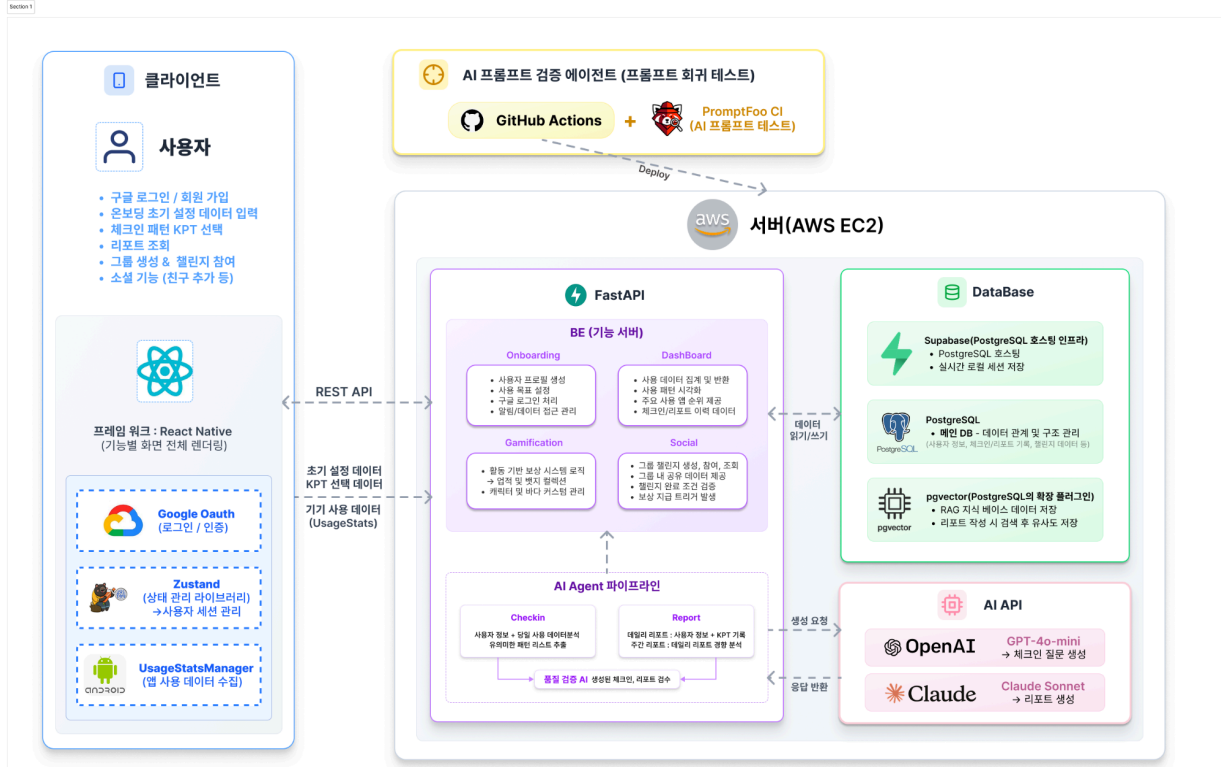
(4) CI/CD — GitHub Actions + PromptFoo

본 항목은 일반적인 코드 빌드·테스트 자동화에 더해, **AI 프롬프트 회귀 테스트** 자동화 계층을 추가로 구축하기 위해서 도입했다. LLM 기반 시스템에서는 프롬프트의 미세한 변경이 출력 품질의 의도하지 않은 회귀를 유발할 수 있으므로, 일반 단위 테스트만으로는 품질을 검증하기 어려울 것으로 판단했다.

이를 해결하기 위해 LLM 출력 평가 전용 도구로 **PromptFoo**를 활용하여 사전 정의된 테스트 케이스에 대해 변경된 프롬프트의 출력을 자동 평가하고, **GitHub Actions**가 PR 시점에 이 평가를 트리거하는 구조로 구축한다. 평가 대상 경로는 “DPP_AI/prompts/” 및 “DPP_AI/services/”이며, 회귀 테스트 통과 여부가 merge 가능 조건에 포함된다. 이 인프라는 앞서 정의한 **품질 검증 요구사항(기능 요구사항 ⑦)** 이 1회성 검증에 그치지 않고 개발 사이클 전반에서 작동하도록 보장하기 위해 결정하였다. 이는, AI 기반 서비스 개발의 본질적 도전 과제 중 하나인 **출력 품질 유지를 도울 것**으로 기대한다.

3.2 전체 시스템 구성

돌핀팻의 전체 시스템은 클라이언트, 서버(BE + AI 파이프라인), 데이터베이스, AI API, CI/CD 5개 레이어로 구성된다.



돌핀팻의 시스템은 데이터 처리 과정에 AI 활용 여부에 따라 두 개의 파이프라인으로 분리하여 설계하였다.

P1. AI 파이프라인: 사용 데이터 수집-AI 기반 인사이트 생성-품질 검증까지를 담당하는 데이터 처리 흐름

1. 체크인 모듈
2. 리포트 모듈

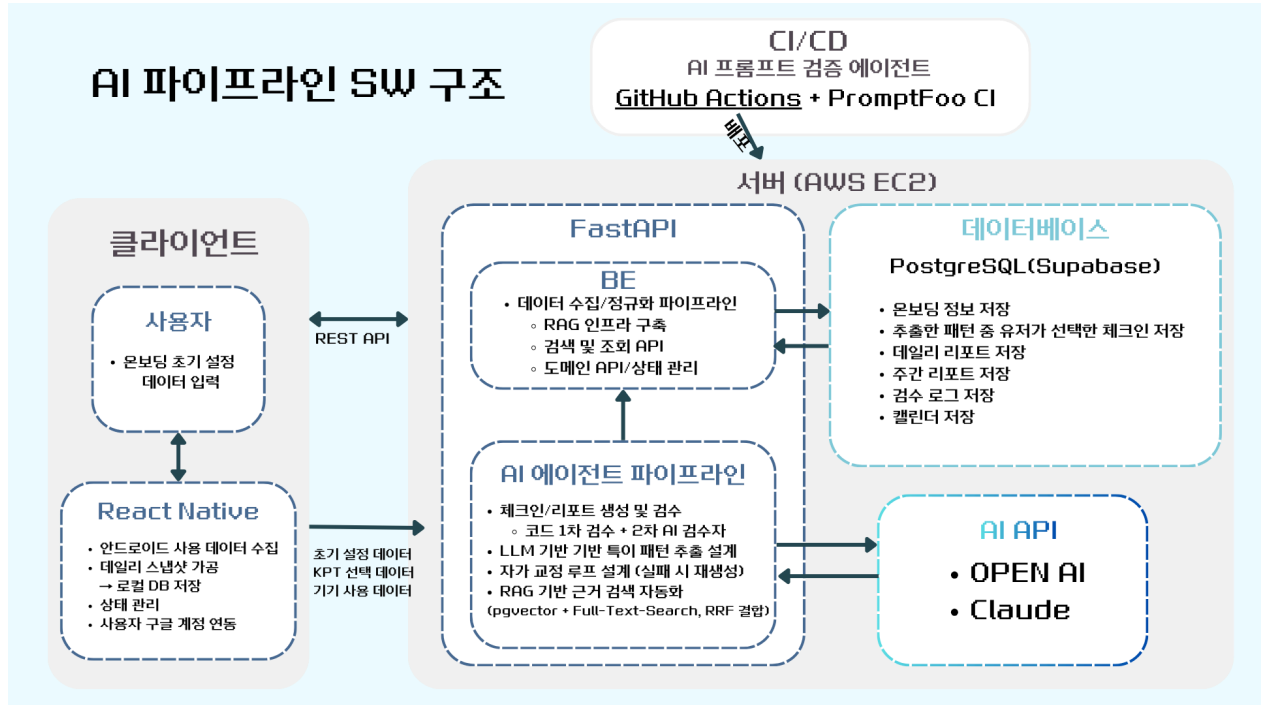
P2. 사용 습관 형성 파이프라인: 대시보드·소셜·게이미피케이션 등 사용자 인터랙션과 동기 부여를 담당하는 사용 습관 형성을 위한 UX 흐름

3. 소셜 모듈
4. 게이미피케이션 모듈

두 파이프라인은 동일한 인프라(AWS EC2 기반 FastAPI 서버, Supabase PostgreSQL, GitHub Actions CI/CD) 위에서 작동하지만, AI 호출 여부에 따라 동작 흐름이 분리되어 있어, 각 파트에서 변경 사항이 발생하더라도 각각의 사용자 경험에 영향을 주지 않으며, 서로 다른 개발 속도로 반복 개발될 수 있다.

P1. AI 파이프라인 SW 구조

AI 파이프라인의 주요 모듈은 1. **체크인 모듈**과 2. **리포트 모듈**로 구성된다. 두 모듈은 RAG 인프라(pgvector + Full-Text-Search 하이브리드 검색)와 2단계 품질 검증 체계(Deterministic + LLM Judge)를 공유하지만, 입력 데이터의 시간 단위·사용 LLM·출력 형태가 서로 다르다.



1. 체크인 모듈

체크인 모듈은 사용자가 일일 사용 패턴을 인식하고 KPT(Keep/Problem/Try) 프레임워크로 회고하도록 돕는 자기인식 메커니즘이다. 본 모듈은 **클라이언트의 데이터 수집 → BE의 정규화 → AI Agent의 생성·검증 → DB 저장**으로 이어지는 단방향 데이터 흐름으로 구성된다. 클라이언트에서 수집한 스마트폰 사용 데이터가 서버로 전달되어 AI Agent의 생성·검증을 거친 뒤, 사용자 회고 결과로 DB에 저장되기까지의 흐름을 담당한다.

모듈 내 데이터 흐름	처리 내용
1. 데이터 수집	[클라이언트] UsageStatsManager API로 당일 사용 데이터 수집 → Daily Snapshot 가공 후 로컬 DB 저장 → REST API로 BE 전송
2. 데이터 정규화	[BE] 데이터 정규화 → AI Agent의 표준 입력 형태로 변환
3. AI 출력 생성	[AI Agent] RAG 검색으로 전문 지식 컨텍스트 구성 → AI API Writer 호출하여 특이 패턴 5개 후보 생성

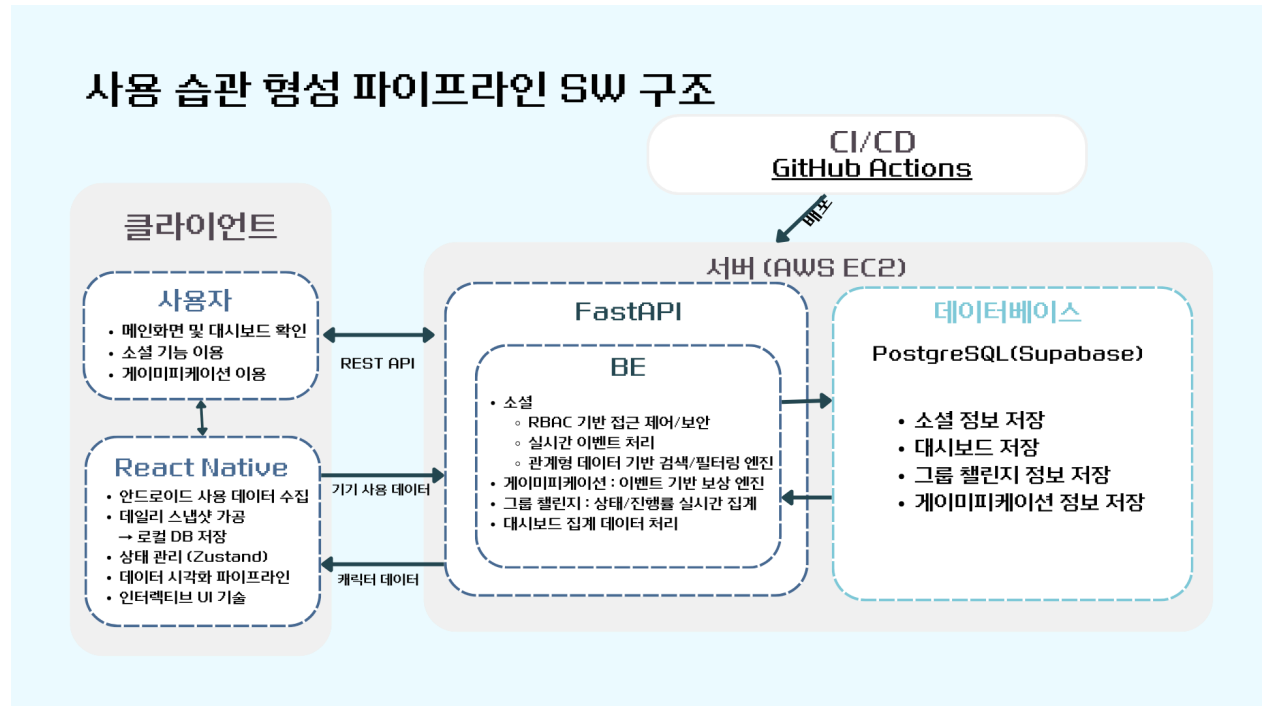
모듈 내 데이터 흐름	처리 내용
4. 출력 결과 검증	[AI Agent] Deterministic 검증 → LLM Judge 검증 실패 시 최대 3회 재시도, 실패 지속 시 사전 정의된 폴백 응답 출력
5. 회고 후 내용 저장	[BE → 클라이언트] 검증 통과한 5개 패턴 후보 반환 → 사용자가 KPT(Keep/Problem/Try) 선택 → BE → DB 저장

2. 리포트 모듈

리포트 모듈은 BE에서 누적된 사용자 데이터를 통합하여 AI Agent로 전달하고, 리포트 생성·검증 결과를 DB에 저장한 뒤 클라이언트로 반환하는 흐름을 담당한다. RAG 인프라와 검증 체계를 체크인 모듈과 공유한다.

체크인 모듈 내 단계	처리 내용
1. 데이터 통합	[BE] 누적된 KPT 기록, Daily Snapshot, 온보딩 시 수집한 사용자 컨텍스트 3종류를 통합
2. AI 출력 생성	[AI Agent] RAG 검색으로 전문 지식 컨텍스트 구성 → AI API Writer 호출 → 데일리/주간 리포트 생성
3. AI 출력 검증	[AI Agent] Deterministic 검증 → LLM Judge 검증 (실패 시 3회까지 재시도 또는 폴백)
4. 저장	[AI Agent → DB] 검수 로그 저장 → 리포트 저장 → 캘린더 인덱스 갱신
5. 리포트 반환	[BE → 클라이언트] 검증 통과한 리포트 반환 → 사용자가 데일리/주간 리포트 또는 캘린더 조회를 통해 열람

P2. 사용 습관 형성 파이프라인 SW 구조



사용 습관 형성 파이프라인의 주요 모듈은 3. **소셜 모듈**과 4. **게이미피케이션 모듈**로 구성된다. 두 모듈은 모두 **이벤트 기반**으로 작동하며, 클라이언트의 사용자 액션이나 다른 모듈로부터 발생한 이벤트를 트리거로 하여 BE의 비즈니스 로직 처리 → DB 갱신 → 클라이언트 동기화로 이어지는 구조를 공유한다.

3. 소셜 모듈

소셜 모듈은 친구 관계 형성, 그룹 챌린지 운영, 응원 알림 등 사용자 간 상호작용을 통해 지속적 동기 부여 환경을 조성하는 모듈이다. 본 모듈은 **클라이언트의 사용자 액션 → BE의 권한 검증·비즈니스 로직 처리 → DB 갱신·실시간 집계 → 알림·결과 반영**으로 이어지는 양방향 이벤트 흐름으로 구성된다. 한 사용자의 액션이 본인뿐 아니라 관계된 다른 사용자의 클라이언트로도 전파된다는 점에서 AI 파이프라인의 단방향 흐름과 구분된다.

모듈 내 데이터 흐름	처리 내용
1. 액션 발생	[클라이언트] 사용자가 친구 신청·수락·삭제, 챌린지 생성·참여·조회, 응원 보내기 등 소셜 액션 수행 → REST API로 BE 전송
2. 권한 검증	[BE] RBAC 기반 접근 제어로 액션 권한 확인 (친구 관계 여부, 챌린지 멤버십, 응원 자격 등)
3. 유형별 로직 처리	[BE] 액션 유형별 로직 처리 → 친구 관계 갱신 or 챌린지 상태·진행률 갱신 / 응원 이벤트 발행

모듈 내 데이터 흐름	처리 내용
4. 실시간 동기화, 저장	[BE → DB] 변경된 관계·챌린지·응원 데이터 저장 → 그룹 챌린지 참여자 진행률 실시간 집계
5. 알림 및 반영	[BE → 클라이언트] 액션 수행자에게 결과 반환 + 관계된 상대 사용자에게 푸시 알림 발송

4. 게이미피케이션 모듈

게이미피케이션 모듈은 사용자의 행동에 대한 보상 체계를 운영하는 모듈이다. 본 모듈은 **다른 모듈로부터의 이벤트 수신 → BE의 보상 규칙 평가·지급 → DB 갱신 → 클라이언트 반영**으로 이어지는 이벤트-보상 흐름으로 구성된다. 트리거가 본 모듈 외부(체크인 모듈, 소셜 모듈)에서 발생한다는 점에서, 다른 모듈의 행동에 의존하는 후속 보상 모듈로 작동한다.

모듈 내 데이터 흐름	처리 내용
1. 이벤트 수신	[BE] 체크인 완료, 챌린지 달성 등 외부 모듈의 행동 이벤트 수신 또는 사용자의 캐릭터 커스터마이징 구매 액션 수신
2. 보상 평가	[BE] 이벤트 유형별 보상 규칙 적용 → 코인 지급 액수 산정, 업적 뱃지 부여 조건 충족 여부 평가
3. 보상 지급	[BE] 코인 가산/차감, 업적 뱃지 부여, 캐릭터 커스터마이징 인벤토리 갱신
4. 상태 저장	[BE → DB] 보유 코인, 획득 뱃지, 캐릭터 커스터마이징 상태 갱신
5. 클라이언트 반영	[BE → 클라이언트] 갱신된 보상 상태 반환 → 클라이언트에서 획득 알림 표시 및 UI 업데이트

클라이언트 (React Native)

구성요소	역할
React Native	기능별 화면 전체 렌더링
Google OAuth	로그인/인증
Zustand	상태 관리 > 사용자 세션 관리
UserStatsManager	앱 사용 데이터 수집

클라이언트는 REST API를 통해 서버와 통신하며, 초기설정 데이터, KPT 선택 데이터, 기기 사용 데이터(UsageStats)를 전달한다.

서버 (AWS EC2 — FastAPI)

기능 서버 (BE)

모듈	주요 역할
Onboarding	사용자 프로필 생성, 사용 목표 설정, 구글 로그인 처리, 알림/데이터 접근 관리
DashBoard	사용 데이터 집계 및 반환, 사용 패턴 시각화, 주요 사용 앱 순위 제공, 체크인/리포트 이력
Social	그룹 챌린지 생성·참여·조회, 그룹 내 공유 데이터 제공, 챌린지 완료 조건 검증, 보상 지급 트리거
Gamification	활동 기반 보상 시스템, 업적·배지 컬렉션, 캐릭터·바다 커스텀 관리

AI Agent 파이프라인

파이프라인	역할
Checkin	사용자 정보 + 당일 사용 데이터 분석 → 유의미한 패턴 리스트 추출
Report	데일리: KPT 기록 기반 생성 / 주간: 데일리 리포트 경향 분석 기반 생성
품질 검증 시	생성된 체크인·리포트 검수

데이터베이스

구성요소	역할
Supabase (PostgreSQL 호스팅)	PostgreSQL 호스팅, 실시간 로컬 세션 저장
PostgreSQL	메인 DB — 사용자 정보, 체크인/리포트 기록, 챌린지 데이터 등
pgvector (PostgreSQL 확장)	RAG 지식 베이스 데이터 저장, 리포트 작성 시 유사도 검색

AI API

API	역할
GPT-4o-mini (OpenAI)	체크인 패턴 후보 생성 및 LLM Judge
Claude Sonnet (Anthropic)	AI 리포트 생성 및 품질 검수

CI/CD

GitHub Actions + PromptFoo CI 활용 -> AI 프롬프트 검증 에이전트 자동화 및 배포

3.3 주요 엔진 및 기능 설계

본 장은 DolphinPod의 기능 구현을 위해 도입된 SW 모듈과 엔진을 기술한다. 각 기능별로 어떤 모듈을 도입하고 어떻게 구현하였는지를 모듈 단위로 상세히 서술한다.

3.3-1 메인화면 / 대시보드 (DashBoard)

기능 개요

대시보드 기능의 구현을 위하여 **Android UsageStatsModule(Native Bridge)**, **DashboardScreen.tsx(FE)**, **dashboard.py 엔드포인트(BE)** 세 모듈을 도입하고, UsageLog ORM 모델을 구현하였다. 이 기능은 사용자가 오늘의 스마트폰 사용 현황을 **수치·시각화·앱 랭킹 3가지 형태로 즉각 인지**할 수 있도록 설계된다.

레이어	모듈	담당 기능
Android Native	UsageStatsModule.kt	UsageStatsManager API 래핑 → React Native Bridge로 앱별 통계 전달
FE	DashboardScreen.tsx	메인 화면 렌더링 / API 호출 / 4개 UI 블록 데이터 바인딩
FE	DashboardAppIcon.tsx	패키지명 기반 앱 아이콘 로딩 / 폴백 UI 처리
BE	endpoints/dashboard.py	GET /api/v1/dashboard — 집계 데이터 조합 후 JSON 반환
BE	models/usage_log.py	UsageLog ORM 스키마 정의: package_name, usage_duration, category, timestamps
BE	endpoints/log.py	POST /api/v1/logs — 클라이언트 수집 데이터 수신 및 DB 저장
상태관리	Zustand store	사용자 세션 토큰 보관 / UsageStats 로컬 캐시 관리

A모듈: UsageStatsModule (Android Native)

▶ 기능 A1 — UsageStatsManager API 데이터 수집

UsageStatsModule은 Android OS의 **UsageStatsManager API**를 직접 호출하여 앱별 Foreground 사용 데이터를 수집하는 모듈이다. **A1 수집 기능**의 처리 알고리즘은 다음과 같다.

① **권한 확인**: 앱 시작 시 AppOpsManager.OPSTR_GET_USAGE_STATS 권한 보유 여부를 확인하고, 없을 경우 PermissionScreen으로 사용자를 유도한다.

- ② **쿼리 범위 설정:** 오늘 00:00:00 ~ 현재 시각을 쿼리 범위로 설정하여 일간 데이터만 수집한다 (UsageStatsManager.INTERVAL_DAILY).
- ③ **통계 집계:** 반환된 UsageStat 리스트를 순회하며 totalTimeInForeground를 기준으로 앱별 사용 시간을 초 단위로 변환·누산한다.
- ④ **Bridge 전달:** React Native WritableMap에 각 앱의 packageName, usageDuration, launchCount, lastTimeUsed를 직렬화하여 JS 레이어로 전달한다.

```
// UsageStatsModule.kt 핵심 로직 (구조 요약)
val usm = ctx.getSystemService(Context.USAGE_STATS_SERVICE) as UsageStatsManager
val stats = usm.queryUsageStats(
    UsageStatsManager.INTERVAL_DAILY, startOfDay, System.currentTimeMillis())
stats.filter { it.totalTimeInForeground > 0 }
    .sortedByDescending { it.totalTimeInForeground }
    .forEach { stat ->
        map.putString("packageName", stat.packageName)
        map.putDouble("usageDuration", stat.totalTimeInForeground / 1000.0)
    }
```

설계 판단: UsageStatsManager는 앱 차단 없이 순수 관찰 데이터만 수집하므로, '차단이 아닌 자기인식'이라는 핵심 철학과 일치한다. 백그라운드 polling 간격은 1분으로 설정하여 배터리 영향을 최소화하면서 실시간성을 확보한다.

▶ 기능 A2 — Daily Snapshot 생성 및 서버 업로드

수집된 raw 데이터를 **세션 단위로 전처리(1차 가공)**하여 서버에 업로드하는 기능이다.

A2 처리 알고리즘:

- ① **시간대 버킷 분류:** 각 앱의 lastTimeUsed 타임스탬프를 기준으로 morning(06~11시), afternoon(12~17시), evening(18~21시), night(22~05시) 4구간에 누산한다.
- ② **핵심 집계값 계산:** total_usage_sec(전체 합산), unlock_count(잠금 해제 횟수), max_continuous_sec(최장 연속 사용), app_launch_count(총 실행 횟수)를 계산한다.
- ③ **역등성 보장:** 서버는 (user_id, date) 복합키 기준 upsert를 수행하므로, 동일 날짜 재전송 시 중복 없이 갱신된다.

```
// API Spec — POST /api/v1/snapshots Request Body
{
  "date": "2026-04-15",
  "timezone": "Asia/Seoul",
```

```

"total_usage_sec": 14400,
"unlock_count": 72,
"time_of_day_buckets_sec": {
  "morning": 1800, "afternoon": 4200,
  "evening": 5400, "night": 3000
},
"max_continuous_sec": 2700,
"app_launch_count": 95,
"schema_version": "1.0.0"
}

```

B모듈: dashboard.py 엔드포인트 (BE)

▶ 기능 B1 — 대시보드 집계 데이터 반환

dashboard.py는 **GET /api/v1/dashboard** 엔드포인트를 구현하는 BE 모듈이다. 이 모듈의 B1 기능은 DB에 누적된 UsageLog 데이터를 오늘 날짜 기준으로 집계하여 FE가 요구하는 형태의 JSON을 구성·반환하는 것이다.

B1 처리 알고리즘:

- ① **사용자 인증:** JWT 토큰에서 user_id를 추출하여 해당 사용자의 데이터만 조회한다.
- ② **오늘 스냅샷 조회:** DailySnapshot 테이블에서 today_date 기준 레코드를 조회하고, 없으면 빈 응답을 반환한다.
- ③ **앱 랭킹 집계:** UsageLog 테이블에서 오늘 날짜의 레코드를 usage_duration 내림차순으로 정렬 후 상위 5개를 추출한다.
- ④ **체크인 이력 조회:** DailyCheckin 테이블에서 오늘 체크인 완료 여부를 조회하여 FE의 동적 버튼 전환("체크인" / "리포트") 근거 데이터를 제공한다.

```

# endpoints/dashboard.py 응답 구조 (간략화)
@router.get('/dashboard')
async def get_dashboard(user=Depends(get_current_user), db=Depends(get_db)):
    snapshot = db.query(DailySnapshot)
    .filter_by(user_id=user.id, date=today).first()
    top_apps = db.query(UsageLog)
    .filter_by(user_id=user.id, date=today)
    .order_by(UsageLog.usage_duration.desc()).limit(5).all()
    checkin_done = db.query(DailyCheckin)

```



```

.filter_by(user_id=user.id, date=today).first() is not None
return DashboardResponse(
    total_usage_sec=snapshot.total_usage_sec,
    unlock_count=snapshot.unlock_count,
    max_continuous_sec=snapshot.max_continuous_sec,
    app_launch_count=snapshot.app_launch_count,
    time_of_day_buckets=snapshot.time_of_day_buckets_sec,
    top_apps=top_apps, checkin_done=checkin_done
)

```

C모듈: DashboardScreen.tsx (FE)

▶ 기능 C1 — 4개 UI 블록 렌더링

DashboardScreen.tsx는 대시보드의 메인 화면 컴포넌트로, **4개의 독립 UI 블록**을 ScrollView 안에 배치하여 사용자에게 오늘 현황을 시각화한다.

블록	컴포넌트	구현 방식
A	핵심 지표 카드	BE 응답의 total_usage_sec·unlock_count·max_continuous_sec·app_launch_count를 초→분 변환 후 4칸 가로 카드에 바인딩
B	시간대 막대 차트	time_of_day_buckets 4구간을 ReportVisuals.tsx의 TimelineBarChart 컴포넌트에 전달하여 수평 막대 시각화
C	상위 앱 랭킹	top_apps 배열을 map()으로 순회하며 DashboardAppIcon.tsx에 packageName 전달 → 아이콘 + 사용시간 표시
D	체크인/리포트 버튼	Zustand store의 checkinDone 상태값 조건부 렌더링 → false: '체크인 하러 가기', true: '오늘 리포트 보기' 버튼 전환

▶ 기능 C2 — DashboardAppIcon 폴백 처리

DashboardAppIcon.tsx는 Android 네이티브 모듈을 통해 패키지명으로 앱 아이콘 바이너리를 요청한다. **C2**

폴백 알고리즘: 아이콘 로딩에 실패하거나 시스템 앱(com.android.*)인 경우 회색 원형 플레이스홀더를 렌더링한다. 이미지는 Base64 인코딩 후 Image 컴포넌트의 source.uri로 전달하여 JS 레이어에서 처리한다.

3.3-2 체크인 / AI 리포트

기능 개요

1. 체크인: 사용 로그 데이터를 정제하여 KPT(Keep, Problem, Try) 패턴 리스트를 뽑아내는 과정이다. 단순한 원시 데이터가 아닌, 스스로 돌아볼 수 있는 의미 있는 패턴을 발견하도록 돕기 위해 설계되었다.
2. AI 리포트: 사용자가 회고하고 싶은 패턴과 온보딩 정보를 바탕으로, 전문 자료에 근거한 코멘트와 시각화된 오늘의 사용량 분석을 포함한 리포트를 생성하는 과정이다. 데일리 리포트가 생성되고 한 주가 지나면, 주간 리포트를 통해 (그 주의) 피드백과 인사이트 확인이 가능하다. 이를 통해 개인화된 피드백과 시각적 분석으로 사용자의 자기 인식과 지속적인 개선을 유도하고자 한다.

기능 관련 모듈

레이어	모듈	담당 기능
Android Native	UsageStatsModule.kt	UsageStatsManager API → React Native Bridge로 앱별 통계 전달
BE 모델	app/models/calendar.py	CheckIn, PatternCandidatesDaily, PatternCandidatesLog
BE 모델	app/models/reports.py	DailyReports, WeeklyReports, ReportDraft, ReportEvidenceTrace, ReportReviewLog
BE 라우터	endpoints/checkin.py	패턴 후보 저장/조회, 체크인 저장/조회
BE 라우터	endpoints/report.py	데일리 리포트 생성/조회
BE 라우터	endpoints/calendar.py	캘린더 조회
BE	AI 클라이언트 services/ai_client.py	AI 서비스 HTTP 호출 (체크인/리포트 파이프라인)
AI	routers/checkin_pipeline	패턴 후보 생성 → 검증 → 판정
BE & AI	리포트 파이프라인 services/report_pipeline_service.py routers/report_pipeline.py	- RAG 기반 근거 수집, AI 호출 및 결과 처리 - 리포트 작성 → 리포트검수
AI	프롬프트 - prompts/	패턴/리포트 생성 관련 ai 가이드라인
AI	deterministic_check.py checkin_writer.py report_writer.py llm_judge.py	AI 체크인 및 리포트 생성 파이프라인 : 검수와 자가 교정

D모듈: AI 체크인 엔진 (FastAP/AI Agent)

▶기능 D1 듀얼 에이전트 기반 특이 패턴 추출

D1 기능은 수집된 데일리 스냅샷과 사용자 프로필을 결합하여 분석하고, 생성자와 검수자 에이전트의 협업을 통해 신뢰성 높은 회고 데이터를 도출하는 모듈이다. 처리 알고리즘은 다음과 같다.

- ① **데이터 결합**: Android UsageStats API로부터 수집된 데일리 스냅샷과 사용자의 온보딩 프로필 정보를 결합하여 분석 컨텍스트를 형성한다.
- ② **패턴 추출 (checkin_writer)**: 생성자 에이전트가 LLM을 기반으로 평소와 다른 앱 사용 시간 급증이나 집중력 저해 구간 등 유의미한 특이 패턴 리스트를 추출한다.
- ③ **심층 검수 (llm_judge)**: 검수자 에이전트가 설정된 논리 기준에 따라 추출된 패턴의 타당성을 검토하고, 근거 반환과 함께 통과 여부를 결정한다.
- ④ **자가 교정 (Self-Correction)**: 검수 실패 시 실패 사유를 피드백으로 담아 생성자에게 전달하며, 최대 3회까지 재생성 루프를 수행한다. 최종적으로, 검수자를 통과한 정제된 패턴 리스트만이 유저에게 제공된다. 이후, 사용자는 추출된 패턴에서 KPT 패턴을 선택하여 저장할 수 있다.

```
# 체크인 에이전트 파이프라인
async def extract_patterns(snapshot, profile):
    # 1단계: 생성자 에이전트 호출
    raw_patterns = await creator_agent.generate(snapshot, profile)
    # 2단계: 검수자 에이전트 호출 및 자가 교정
    for retry in range(3):
        validation = await inspector_agent.validate(raw_patterns)
        if validation.is_passed:
            return raw_patterns
        raw_patterns = await creator_agent.regenerate(validation.feedback)
```

설계 판단: AI의 할루시네이션(Hallucination)을 방지하기 위해 생성과 검수 역할을 분리한 듀얼 에이전트 구조를 채택했다. 이는 단순한 데이터 요약을 넘어 사용자가 스스로의 행동을 객관적으로 인식하게 돕는 정교한 '자기인식 도구'로서의 신뢰성을 확보하기 위함이다.

E모듈: AI 리포트 엔진 (Supabase / RAG)

▶ 기능 E2 — 하이브리드 RAG 및 근거 검색

E2 기능은 전문 가이드와 사용자의 기록을 결합하여 근거 중심의 피드백을 생성하기 위한 지능형 검색 모듈이다. 처리 알고리즘은 다음과 같다.

- ① **임베딩 데이터셋**: 디지털 중독 치료 가이드 등 전문 문서를 청크(Chunk) 단위로 분할하고 벡터화하여 **pgvector**에 저장한다.
- ② **하이브리드 검색**: 사용자의 현재 패턴을 쿼리로 하여 시맨틱 검색(Semantic Search)과 전체 텍스트 검색(Full-Text Search)을 병행 수행한다.
- ③ **RRF 순위 재계산**: 두 검색 결과에 **RRF(Reciprocal Rank Fusion)** 알고리즘을 적용하여 사용자의 상황과 가장 관련성이 높은 전문 근거를 정밀하게 추출한다.
- ④ **AI 리포트 생성**: 추출된 근거와 KPT 기록을 결합하여 행동 심리학적 가이드가 포함된 최종 리포트를 생성하고 검수, 자가 교정 루프 과정을 거친다.

```
# RAG 처리
def run_rag_agent(
    snapshot: Daily_Snapshots,
    checkin: CheckIn,
    user_config: User_Configs | None,
    db: Session,
) -> Dict[str, Any]:

    def _hybrid_search_expert_knowledge(
        db: Session,
        queries: List[str],
        top_k: int = 5,
        rrf_k: int = 60
    ) -> List[Dict[str, Any]]:
        # 1. 벡터 검색 (OpenAI Embedding)
        vector_rows = db.query(ExpertKnowledge)
            .order_by(ExpertKnowledge.embedding.cosine_distance(query_embedding))

        # 2. Full-Text Search (PostgreSQL tsvector)
        fts_rows = db.query(ExpertKnowledge)
            .filter(ts_vector.op("@@")(ts_query))

        # 3. RRF로 결합
        rrf_score = 1.0 / (rrf_k + rank)
```

설계 판단: 단순 LLM 응답의 한계를 극복하기 위해 전문 지식 베이스를 활용한 RAG(검색 증강 생성) 파이프라인을 구축했다. 특히 RRF 결합 검색을 통해 사용자의 미세한 맥락 변화를 정확히 포착하여, 임상적 근거에 기반한 실질적인 개선책을 제시하고자 한다.

D,E 모듈에서 공통된 검수와 자가 교정 과정

사용자의 사용 패턴을 추출하고, 이를 근거 기반의 개인화 리포트로 변환하는 과정에서 AI의 할루시네이션 방지를 위해 생성-검증-교정 과정의 3단계 루프를 거친다.

1단계: 결정론적 검증

생성된 패턴 후보나 리포트 초안은 deterministic_check.py를 통해 자동화된 규칙 기반 검증을 거친다. 이 단계는 JSON 스키마, 금지어, 수치 일관성을 검사하며, 이를 통과하지 못하면 이후 단계로 진행되지 않는다.

2단계: 의미론적 검증

1단계 검증을 통과하면, llm_judge.py를 통해 의미론적 타당성을 AI 기반으로 평가받는다. 이 단계에서는 단순 규칙 위반을 넘어 맥락적 일관성, 근거의 충분성 등을 판단한다.

3단계: 자가 교정 루프

1단계 또는 2단계에서 거절되면, 생성자에게 구체적인 피드백과 함께 재작성을 지시한다. 이 과정을 최대 3회까지 반복되며, 매 회차마다 요구 기준을 점진적으로 강화한다.

F모듈: 주간 리포트 엔진 (Aggregation & Insight)

▶ 기능 F1 - 데이터 어그레이션 및 정량 지표 산출

F1 기능은 일주일간 누적된 데일리 리포트와 체크인 데이터를 통합 분석하여 사용자의 변화를 수치화하는 모듈이다.

- ① **데이터 집합**: 최근 7일간의 체크인 참여 횟수, KPT 선택 이력, 데일리 리포트 핵심 키워드 추출 후 통합한다.
- ② **습관 점수 계산**: 온보딩 시 설정한 목표 달성률과 체크인 성실도를 결합하여 사용자별 맞춤형 습관 점수를 산출한다.
- ③ **추세 분석**: 전주 대비 총 사용 시간 변동 폭 및 행동 패턴의 긍정/부정 변화율을 벡터화하여 분석한다.

```
// API Spec
{
  "week_range": "2026-W15",
  "engagement": { "check_in_count": 6, "completion_rate": 85.7 },
  "quantitative_metrics": {
```

```
"habit_score": 88,  
"usage_trend": -12.5, // 전주 대비 12.5% 감소(긍정)  
"top_problem_pattern": "Night-time YouTube usage"  
},  
"visual_data": { "daily_scores": [70, 85, 90, 60, 95, 80, 88] }  
}
```

▶ 기능 F2 - 전략적 주간 인사이트 및 가이드 생성

F2 기능은 정량적 분석 결과를 바탕으로 한 주를 평가하고, 다음주를 위한 실행 가능한 전략을 제시하는 AI 엔진이다.

- ① **컨텍스트 구성:** 어그리게이션된 수치 데이터와 온보딩 시 등록한 사용자의 장기 목표(예: "시험 기간 집중력 향상")를 LLM의 프롬프트 컨텍스트로 구성한다.
- ② **주간 코멘트 생성:** 한 주간의 주요 성취와 반복된 문제점을 요약하고, 이를 개선하기 위한 구체적인 전략(Try)을 생성한다.
- ③ **회고 피드백:** 사용자가 자신의 변화를 시각적으로 인지하고 자기 효능감을 느낄 수 있도록 격려와 분석이 조화된 톤앤매너로 출력한다.

설계 판단: 데일리 리포트가 '오늘의 반성'이라면, 주간 리포트는 '장기적 행동 교정'을 위한 나침반 행동을 한다. 현재 데이터 어그리게이션을 위한 DB 스키마 설계는 완료되었으며, 후에 추세 분석 알고리즘 고도화와 함께 정식 구현될 예정이다.

3.3-3 소셜 · 게이미피케이션 (Social & Gamification)

기능 개요

소셜·게이미피케이션 기능의 구현을 위하여 **social.py, challenge.py, gamification.py**(BE 모델), **endpoints/social.py, endpoints/gamification.py**(BE 라우터), **SocialScreen.tsx**(FE) 모듈을 도입하고 구현하였다. 이 기능은 **외부 동기(소셜 챌린지)**와 **내적 보상(코인·배지)**을 결합하여 사용자의 습관 변화 지속성을 지원하는 보상 레이어이다.

레이어	모듈	담당 기능
FE	SocialScreen.tsx	소셜 허브 화면 : 친구/응원/챌린지 탭, API 연동
FE	FriendsScreen.tsx	친구 검색·추가 UI — 포스터 세션 전 완료 예정
FE	GroupDetailScreen.tsx	그룹 챌린지 상세 UI — 포스터 세션 전 완료 예정
BE 모델	models/social.py	Friendship, Alert ORM: 친구 관계 상태 관리
BE 모델	models/challenge.py	Challenge, ChallengeParticipant ORM: 챌린지 생명주기
BE 모델	models/gamification.py	Achievement, UserAchievement, CoinTransaction, Character ORM
BE 라우터	endpoints/social.py	친구 CRUD / 응원 알림 / 챌린지 CRUD 엔드포인트
BE 라우터	endpoints/gamification.py	코인 지급·조회, 업적 부여·조회, 캐릭터 커스텀
BE 라우터	endpoints/challenges.py	챌린지 생성·참여·완료 검증·조회 엔드포인트

G모듈: social.py + challenge.py (BE 모델)

▶ 기능 G1 — 친구 관계(Friendship) 상태 기계

social.py는 Friendship 테이블로 사용자 간 친구 관계를 관리한다.

G1 기능의 상태 전이 알고리즘:

```
# models/social.py — Friendship ORM
class Friendship(Base):
    __tablename__ = 'friendships'
```

```

id      = Column(Integer, primary_key=True)
user_id = Column(Integer, ForeignKey('users.id'), nullable=False)
friend_id = Column(Integer, ForeignKey('users.id'), nullable=False)
status  = Column(Enum('pending','accepted','blocked'), default='pending')
created_at = Column(DateTime, default=func.now())
updated_at = Column(DateTime, onupdate=func.now())
# UniqueConstraint(user_id, friend_id) — 중복 요청 방지
__table_args__ = (UniqueConstraint('user_id','friend_id'),)

```

상태 전이 규칙: ① 친구 요청 → pending 레코드 생성, ② 수락 → status='accepted'로 업데이트 후 양방향 레코드 보장, ③ 차단 → status='blocked' 처리 및 응원 알림 발송 제외. endpoints/social.py의 POST /api/v1/social/friends 핸들러가 이 전이 로직을 수행한다.

▶ 기능 G2 — 응원(Cheer) 알림 발송

G2는 체크인 완료 이벤트를 수신하여 친구들에게 응원 알림을 발행하는 기능이다.

처리 알고리즘:

- ① **트리거 감지:** endpoints/checkin.py의 체크인 저장 핸들러 완료 시 social 서비스를 호출한다.
- ② **수신자 조회:** Friendship 테이블에서 status='accepted'인 friend_id 리스트를 조회한다.
- ③ **Alert 레코드 생성:** 각 친구 user_id에 대해 Alert 레코드(type='cheer', from_user_id, message)를 INSERT한다.
- ④ **Push 발송 (예정):** FCM token이 등록된 경우 Firebase Cloud Messaging을 통해 Push Notification을 발송한다. 현재는 Alert DB 저장까지 구현되었으며, FCM 연동은 포스터 세션 전 완료 목표이다.

▶ 기능 G3 — 그룹 챌린지 생명주기 관리

challenge.py는 Challenge와 ChallengeParticipant 두 ORM 클래스로 챌린지 전체 생명주기를 관리한다.

G3 처리 알고리즘:

```

# models/challenge.py — Challenge ORM (핵심 필드)
class Challenge(Base):
    __tablename__ = 'challenges'
    id      = Column(Integer, primary_key=True)
    creator_id = Column(Integer, ForeignKey('users.id'))
    title    = Column(String(200), nullable=False)
    goal_type = Column(Enum('usage_time','unlock_count',
                           'app_launch_count'), nullable=False)
    goal_value = Column(Integer, nullable=False) # 초 또는 횟수
    start_date = Column(Date, nullable=False)

```



```

end_date = Column(Date, nullable=False)
status = Column(Enum('active','completed','failed'), default='active')
class ChallengeParticipant(Base):
    __tablename__ = 'challenge_participants'
    challenge_id = Column(Integer, ForeignKey('challenges.id'))
    user_id = Column(Integer, ForeignKey('users.id'))
    progress = Column(Float, default=0.0) # 0.0~1.0
    completed = Column(Boolean, default=False)
    completed_at = Column(DateTime, nullable=True)

```

챌린지 완료 검증 알고리즘(endpoints/challenges.py): 매일 스케줄러(포스터 세션 전 완료 예정)가 활성 챌린지의 각 참여자에 대해 (실제 사용량 / goal_value)로 progress를 계산하여 업데이트한다. goal_type이 usage_time인 경우 DailySnapshot의 total_usage_sec이 goal_value 이하이면 달성으로 판정하고, ChallengeParticipant.completed=True로 갱신 후 gamification 모듈의 코인 지급 트리거를 호출한다.

H모듈: gamification.py (BE 모델)

▶ 기능 H1 — 코인 트랜잭션 시스템

gamification.py는 CoinTransaction, Achievement, UserAchievement, Character 4개 ORM 클래스로 보상 시스템을 구현한다.

H1 코인 지급 알고리즘:

```

# models/gamification.py — CoinTransaction ORM
class CoinTransaction(Base):
    __tablename__ = 'coin_transactions'
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'))
    amount = Column(Integer, nullable=False) # 양수=지급, 음수=사용
    reason = Column(String) # 'checkin'|'challenge'|'streak'|'achievement'
    created_at = Column(DateTime, default=func.now())

# 현재 잔액 조회: 트랜잭션 누적 합산 방식
# SELECT SUM(amount) FROM coin_transactions WHERE user_id = :uid

```

코인 지급 트리거와 금액은 다음 규칙에 따른다:

트리거	지급량	호출 위치
데일리 체크인 완료	10코인	endpoints/checkin.py save_checkin()
그룹 챌린지 완료	50코인	endpoints/challenges.py complete_challenge()
7일 연속 체크인 스트릭	100코인	BE Scheduler (포스터 세션 전 구현 예정)
업적 최초 획득	30코인	endpoints/gamification.py grant_achievement()

코인 잔액은 별도 balance 컬럼을 두지 않고 **트랜잭션 테이블의 SUM(amount) 집계 방식**으로 계산한다. 이 방식은 지급 이력 감사(Audit) 추적을 용이하게 하며, 중복 지급 방지를 위해 reason + created_at + user_id의 유니크 제약을 체크인 관련 트랜잭션에 적용한다.

▶ 기능 H2 — 업적 뱃지 시스템

H2는 특정 행동 조건 달성 시 업적을 자동 부여하는 기능이다. **Achievement.condition 필드를 JSON 형태로 설계**하여 조건 메타데이터를 유연하게 정의한다.

```
# models/gamification.py — Achievement ORM
class Achievement(Base):
    __tablename__ = 'achievements'
    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    description = Column(String)
    condition = Column(JSON)
    # condition 예시:
    # {'type': 'streak', 'days': 7}
    # {'type': 'checkin_count', 'count': 30}
    # {'type': 'challenge_complete', 'count': 3}

class UserAchievement(Base):
    __tablename__ = 'user_achievements'
    user_id = Column(Integer, ForeignKey('users.id'))
    achievement_id = Column(Integer, ForeignKey('achievements.id'))
    granted_at = Column(DateTime, default=func.now())
```

업적 평가 알고리즘: endpoints/gamification.py의 evaluate_achievements(user_id) 함수는 등록된 모든 Achievement의 condition JSON을 순회하며, condition.type에 따라 해당 사용자의 체크인

이력·챌린지 완료 횟수·스트릭 일수를 조회하여 충족 여부를 판단한다. 미취득 업적이 조건을 충족하면 UserAchievement 레코드를 INSERT하고 H1 코인 트리거를 호출한다.

▶ 기능 H3 — 캐릭터 커스텀 시스템

H3는 코인으로 캐릭터·배경·액세서리를 잠금 해제하고 장착 상태를 관리하는 기능이다.

```
# models/gamification.py — Character ORM
class Character(Base):
    __tablename__ = 'characters'
    user_id = Column(Integer, ForeignKey('users.id'), primary_key=True)
    base_type = Column(String, default='dolphin')
    equipped_items = Column(JSON, default={})
    # equipped_items: {'hat': 'crown', 'bg': 'ocean_sunset', ...}
    unlocked_items = Column(JSON, default=[])
    # unlocked_items: ['crown', 'ocean_sunset', 'star_accessory']
    updated_at = Column(DateTime, onupdate=func.now())
```

H3 커스텀 알고리즘: ① 아이템 구매: 해당 아이템의 cost를 CoinTransaction으로 차감 후 unlocked_items 배열에 추가, ② 장착: equipped_items JSON의 해당 슬롯(hat/bg/accessory)에 아이템 ID를 업데이트, ③ UI 렌더링: FE는 equipped_items를 받아 캐릭터 레이어 이미지를 조합·렌더링한다. 커스텀 화면 UI(FE)는 포스터 세션 전 완료 예정이다.

I모듈: endpoints/social.py + endpoints/gamification.py (BE 라우터)

▶ 기능 I1 — 소셜 API 엔드포인트

endpoints/social.py는 친구 관계·응원·챌린지 관련 REST API를 제공한다. 주요 엔드포인트:

엔드포인트	메서드	기능
/api/v1/social/friends	POST	친구 요청 발송 (pending 레코드 생성)
/api/v1/social/friends/{id}/accept	PATCH	친구 수락 (status → accepted)
/api/v1/social/friends	GET	친구 목록 조회 (accepted 상태만)
/api/v1/social/alerts	GET	응원 알림 이력 조회 (SocialScreen 응원 탭)
/api/v1/challenges	POST	챌린지 생성 + 참여자 초대
/api/v1/challenges/{id}	GET	챌린지 상세 + 참여자 진행률 조회
/api/v1/challenges/{id}/join	POST	챌린지 참여 (ChallengeParticipant INSERT)

▶ 기능 I2 — 게이미피케이션 API 엔드포인트

endpoints/gamification.py는 코인·업적·캐릭터 커스텀 관련 REST API를 제공한다.

엔드포인트	메서드	기능
/api/v1/gamification/coins	GET	현재 코인 잔액 조회 (SUM 집계)
/api/v1/gamification/achievements	GET	취득 업적 목록 조회
/api/v1/gamification/character	GET	현재 캐릭터 장착 상태 조회
/api/v1/gamification/character/equip	PATCH	아이템 장착 (equipped_items 업데이트)
/api/v1/gamification/shop/buy	POST	아이템 구매 (코인 차감 + unlocked 추가)

J모듈: SocialScreen.tsx (FE)

▶ 기능 J1 — 소셜 탭 구조 및 상태 관리

SocialScreen.tsx는 친구·응원·챌린지 세 탭을 포함하는 소셜 허브 화면이다. **J1 처리 방식:** 탭 선택 상태를 useState로 관리하며, 탭 전환 시 해당 탭에 대응하는 API를 useEffect로 호출하여 데이터를 fetch한다. 각 탭의 데이터는 독립 상태(friendList, alertList, challengeList)로 분리하여 불필요한 재렌더링을 방지한다.

J1 탭별 연동 API:

- 친구 탭: GET /api/v1/social/friends → 수락된 친구 목록 표시
- 응원 탭: GET /api/v1/social/alerts → 수신한 응원 이력 시간순 표시
- 챌린지 탭: GET /api/v1/challenges?user_id=me → 참여 중인 챌린지 + 진행률 표시

3.4 주요 기능의 구현

본 장은 3.3에서 기술한 모듈이 기능 단위에서 어떻게 연관·통합되는지를 서술한다. 각 기능별로 도입된 모듈 간의 데이터 흐름과 처리 알고리즘을 중심으로 기술한다.

3.4-1 대시보드 기능과 SW 모듈 연관성

대시보드 기능은 **Android UsageStatsModule → BE log.py → BE dashboard.py → FE DashboardScreen.tsx**의 4단계 파이프라인으로 구현된다. FE의 DashboardScreen.tsx는 Zustand store에서 JWT 토큰을 획득하여 GET /api/v1/dashboard를 호출하며, BE의 dashboard.py는 UsageLog ORM에 누적된 데이터를 집계하여 응답한다. 대시보드 집계 품질은 AI 체크인 파이프라인의 입력 품질에 직결된다.

단계	주체	처리 내용	모듈
①	Android Native	UsageStatsManager API → 앱별 사용 통계 수집 (1분 polling)	UsageStatsModule.kt
②	FE → BE	Daily Snapshot 전처리 (시간대 버킷 분류·핵심 지표 계산) 후 업로드	POST /api/v1/snapshots
③	BE	UsageLog ORM으로 PostgreSQL 저장 (upsert — (user_id, date) 복합키)	models/usage_log.py
④	BE → FE	집계 데이터(4개 지표 + 상위 앱 5개 + 시간대 버킷) JSON 반환	GET /api/v1/dashboard
⑤	FE	Zustand 상태관리 → 4개 UI 블록 데이터 바인딩 및 렌더링	DashboardScreen.tsx

Android 기기에서 앱별 사용 통계를 수집하고, 이를 서버에 저장 및 가공하여 사용자에게 대시보드 형태로 제공하는 5단계로 구성되어 있다. 전체 흐름은 수집(Native) → 전처리(FE) → 저장(BE) → 집계(BE) → 표시(FE) 순으로 진행된다.

3.4-2 소셜·게이미피케이션과 SW 모듈 연관성

소셜·게이미피케이션은 **체크인·챌린지 완료 이벤트**를 트리거로 하는 보상 레이어로, 독립 기능이 아니라 기존 파이프라인에 연결된 사이드 이펙트 구조로 설계된다.

트리거 기능	소셜 연관 처리	게이미피케이션 연관 처리
데일리 체크인 완료	친구 Alert 레코드 생성 → FCM Push 발송(예정) → SocialScreen 응원 탭 갱신	10코인 CoinTransaction INSERT → 스트릭 카운트 업 → 업적 평가기 호출

그룹 챌린지 달성	ChallengeParticipant.completed=True → 그룹원 전원에게 완료 알림 발송	50코인 CoinTransaction INSERT→ '소셜 리더' 업적 카운트 평가
AI 리포트 조회	주간 리포트 인사이트 기반 그룹 챌린지 목표 자동 제안(예정)	'패턴 분석가' 업적 카운트 평가

3.4-3 구현 전체 개요 및 기능별 검증 결과

현 단계에서는 에뮬레이터 기반 전체 데이터 흐름 검증을 완료하였으며, Android 실기기 테스트 및 데이터 누적을 진행 중이다. **포스터 세션 전 실기기 통합 검증을 완료**하는 것을 목표로 한다.

구현 기능	검증 방법	결과 / 현황
UsageStats 수집 정확도	에뮬레이터에서 앱 강제 실행 후 UsageStats 집계값과 실제 사용 시간 비교	에뮬레이터 기준 정합성 확인. 실기기 데이터 누적 후 정밀 검증 예정
dashboard.py 집계 응답 검증	Postman으로 GET /api/v1/dashboard 호출 후 응답 JSON 스키마·값 검증	4개 지표·상위앱·시간대 버킷 정상 반환 확인
챌린지 생성·참여 API	POST /api/v1/challenges → ChallengeParticipant 레코드 생성 DB 직접 조회	관계 테이블 정상 생성 확인. FE 연동 UI 포스터 세션 전 검증 예정
코인 지급 트리거 정확성	체크인 완료 API 호출 후 coin_transactions 테이블 자동 INSERT 확인	10코인 자동 지급 정상 동작 확인 (reason='checkin')
업적 평가 로직 단위 테스트	evaluate_achievements() 함수에 mock 사용자 데이터 주입 후 조건 충족 판정 확인	streak·checkin_count 조건 정상 판정. challenge 조건 구현 중
소셜 응원 알림 DB 저장	체크인 완료 시 Alert 레코드 자동 생성 여부 조회	Alert DB 저장 정상 확인. FCM 발송은 포스터 세션 전 실 디바이스 테스트 예정

3.5 기타 : AI 프롬프트 설계, 테스트 후 개선 내용

돌핀팻 서비스에서 AI 에이전트 위상

본 시스템은 LLM 기반 콘텐츠 생성에 의존하므로, **AI 에이전트**는 단순 구현 디테일이 아닌 **시스템 동작 사양의 일부**로 취급된다. 프롬프트의 미세한 변경이 출력 품질에 직접적인 영향을 미치는 LLM 시스템의 특성상, 본 프로젝트는 프롬프트를 코드와 동일한 수준에서 버전 관리하고 있다. 또한, PromptFoo 기반 자동 회귀 테스트를 진행하여 품질을 관리하고자 한다.

본 시스템의 주요 AI 에이전트 (프롬프트)는 다음 세 가지다.

에이전트 종류	적용 LLM 모델	역할
체크인 Writer	OpenAI GPT-4o-mini	일일 사용 패턴 5개 후보 자연어 생성
리포트 Writer	Anthropic Claude Sonnet	데일리·주간 개인화 리포트 생성
LLM Judge	OpenAI GPT-4o-mini	Writer 출력에 대한 정성 품질 평가

프롬프트 핵심 설계 원칙

본 프로젝트의 프롬프트는 다음 원칙을 공통적으로 따른다.

- **단정·평가·조언의 명시적 금지:** 본 서비스의 핵심 철학("차단이 아닌 자기인식")에 따라 AI는 사용자의 행동을 의학적으로 진단하거나 단정적으로 평가하지 않는다.
- **데이터 기반 서술의 강제:** 입력 데이터에 존재하는 수치만 사용하도록 제약하여 AI 환각 현상(hallucination)을 차단한다.
- **표기 규칙의 일관성:** 시간 단위(59초 이하 생략, 60초 이상은 분 단위 올림) 등을 프롬프트 차원에서 강제한다.
- **출력 형식의 엄격한 제약:** JSON 스키마를 명시하고 그 외 텍스트 출력을 금지하여 후속 처리의 안정성을 확보한다.

AI 프롬프트 테스트 및 개선 사례

프롬프트는 운영 과정에서 발견된 문제에 따라 반복적으로 개선된다. 본 프로젝트에서는 다음 두 가지 주요 사항에 대해 테스트 후 개선이 이루어졌다.

(1) 체크인 Writer 프롬프트의 임상 용어 사용 문제 (v1 → v2)

문제 발견

초기 v1 프롬프트로 운영 시, RAG 지식 베이스에서 검색된 임상·치료 관련 콘텐츠(cbt_ia 카테고리)를 컨텍스트로 주입했을 때 Writer가 "중독", "의존성", "CBT" 등 임상 용어를 그대로 사용자에게 노출하는 문제가 발견되었다. 이는 본 서비스의 진단 금지·중립적 톤 원칙에 위배되므로 다음과 같이 수정하였다.

v2 개선 내용

개선 항목	v1	v2
retrieved_evidence 처리 규칙	명시적 규칙 없음	"배경 원리 이해에만 활용, 사용자 노출 문장에는 임상 용어 직접 사용 금지" 명시
중립적 재서술 가이드	없음	변환 예시 제공 (예: "중독적인 사용 패턴" → "특정 시간대에 집중되는 사용 리듬", "앱 의존성" → "자주 열게 되는 앱 패턴")
조언형 표현 차단	일반적 금지 문구만	"~하세요", "권장합니다", "줄이세요" 등 구체적 금지 표현 명시

이 개선을 통해 Writer 출력이 RAG 지식 베이스의 출처와 무관하게 일관된 톤을 유지하게 되었다.

(2) LLM Judge 프롬프트의 오탐 문제 (v1 → v2)

문제 발견

초기 v1 Judge 프롬프트는 정성 기준(advice_present, judgmental_tone)에 대한 판단 기준이 추상적이어서, Writer가 단순히 임상 출처(cbt_ia)를 인용했다는 사실만으로 Judge가 "조언이다", "비판적이다"라고 잘못 판정하여 불필요한 AI 호출 재시도가 반복되었다. 이는 정성 기준에 구체적 예시가 없을 때 발생하는 LLM 정성 평가 특유의 비결정성 문제였음을 알게 되었다.

v2 개선 내용

v2 프롬프트는 다음 세 가지 측면에서 판단 기준을 구체화하였다.

- **정성 기준의 예시 명시:** `advice_present`가 true인 경우와 false인 경우의 구체적 예시 문장을 양쪽 모두 제공
 - true 예시: "~하세요", "줄이세요", "해야 합니다", "권장됩니다"
 - false 예시: "연구에서 언급된다", "문헌상 ~로 보고된다", "자료에서 ~한 점이 알려져 있다"
- **출처 주제와 톤의 분리:** 출처가 임상·치료 논문이라도 Writer가 단순 사실 서술로 인용한 경우는 위반으로 보지 않음을 명시
- **직접 진단의 정의 명확화:** "당신은 ~중독입니다"와 같이 사용자에게 질병·장애를 직접 단정하는 문장만이 의학적 진단 위반에 해당하며, 일반적 연구 인용은 진단이 아님을 분리하여 명시

이 개선 결과, Judge의 오탐으로 인한 불필요한 AI 호출 재시도 빈도가 감소하였고, 동일한 입력에 대한 Judge 판정의 안정성이 향상되었다.

프롬프트 테스트 및 개선 과정의 의의

위 두 사례는 본 프로젝트가 LLM 시스템 개발에서 마주치는 본질적 도전, 즉 **(1) 외부 지식 베이스로부터의 의도하지 않은 톤 유입**과 **(2) LLM 정성 평가의 비결정성** 문제에 대해 어떻게 대응했는지를 보여준다.

특히 두 번째 개선 과정에서는 **정성적으로 프롬프팅한 기준은 추상적 명령만으로는 LLM에 안정적으로 전달되지 않으며, true/false 양쪽의 구체적 예시가 함께 제공되어야 결과가 유의미하게 변화한다**는 점을 배우고, 에이전트의 품질을 더욱 향상할 수 있었다. 이후 이 규칙은 본 프로젝트의 모든 프롬프트 작성에 일관되게 적용되고 있다.

프롬프트 품질 관리 인프라

프롬프트는 `DPP_AI/prompts/` 디렉토리에서 별도 버전 관리되며, 변경 시 다음 자동화된 검증 절차를 거친다.

1. **PromptFoo 회귀 테스트:** 사전 정의된 테스트 케이스에 대해 변경된 프롬프트의 출력을 자동 평가
2. **GitHub Actions 트리거:** PR(Pull Request) 시점에 회귀 테스트가 자동 실행
3. **merge 조건 강제:** 회귀 테스트 통과를 공동 github 브랜치 merge 가능 조건에 포함

이 인프라는 프롬프트의 변경이 출력 품질에 의도하지 않은 영향을 미치는 것을 사전에 방지하며, **3.1 기능 요구사항의 품질 검증 항목**의 각 검증 단계들이 단발적인 검증에 그치지 않고 개발 사이클 전반에서 작동하도록 보장하는 운영적 기반이 된다.

3.6 주요 개발 현황

핵심 검증 사항1. AI 멀티 에이전트 파이프라인 구현 및 검증 (80%)

가장 핵심 엔진인 AI 파이프라인에 대해 다음과 같은 기술 검증을 완료하였다. 앞으로 api 연동이 모두 완료됨에 따라 목업 데이터를 점차 실기기 데이터로 변환하여 검증하며 고도화해갈 예정이다.

WF1. 사용량 계산 및 스냅샷 생성 기능 : 목업 데이터를 기반으로 구현되었다.

WF2. 체크인 기능 파이프라인 : 위 스냅샷을 2단계 QA 검증하는 구조로 품질을 보증한다.

WF3. 리포트 기능 파이프라인 : RAG-pqvector 연동을 통해 근거 있는 리포트를 생성한다.

WF4. AI 프롬프트 회귀 테스트 시스템 : PromptFoo와 GitHub Actions 기반으로 자동 검증이 실행된다.

핵심 검증 사항2. 주요 화면 구현 (80%)

사용자가 직접 접하게 될 핵심 화면 전반에 대해서 초기 단계로 모두 구현 완료하였다. 기능 개발과 병행하여 점차 고도화해갈 예정이다.

1. 온보딩 : 초기 온보딩 화면, 앱별 카테고리 설정 화면
2. 체크인 : 체크인 질문 생성 화면, 회고 기록 화면
3. 리포트 : 캘린더 화면, 데일리/주간 리포트 화면
4. 게이미피케이션 요소 : 상점 화면, 보상(업적, 뱃지 등) 화면
5. 소셜 : 친구 요청 및 친구 목록 화면, 그룹 바다 화면
6. 메인 화면 : 개인 바다 화면, 대시보드 및 실시간 사용량 화면, 설정 화면

핵심 검증 사항3. 스마트폰 사용 데이터 수집을 위한 API 연결 (70%)

수집된 사용 데이터를 AI 파이프라인 및 백엔드 서버와 연결하는 API 연결을 다음과 같이 완료하였다. API 명세서에 작성한 40개항목 중 완료된 건이 29개로, 현재 진행척도는 70%이다. 앞으로 게이미피케이션 - 인벤토리 및 아이템 관련, 소셜 기능 - 업적 및 그룹 챌린지 등에 필요한 남은 API를 연결 작업을 진행할 예정이다.

1. 온보딩 : 초기 온보딩 정보 저장, 앱별 카테고리 확정, Google OAuth 로그인
2. 체크인 : 데일리 스냅샷 저장, 체크인 질문 패턴 후보 생성 및 조회, 결과 저장 및 조회
3. 리포트 : 대시보드 조회, 데일리/주간 리포트 생성 및 조회
4. 소셜 : 프로필 조회/검색/저장, 친구 리스트 확인, 친구 요청 송수신, 친구 리스트 관리, 친구 응원 기능
5. 메인 화면 : 알림 설정, 주간 목표 설정, 심야 시간 설정, 주요 활동 시간 설정, 체크인 시간 재설정

3.7 핵심 기능의 최종 구현 목표

1. 데일리 체크인

매일 지정 시간에 푸시 알림을 발송하고, 당일 사용 데이터를 세션 단위로 전처리한 후 AI가 특이 사용 패턴 5가지 후보를 자연어 문장으로 제시한다. 사용자는 KPT 형식으로 패턴을 선택하여 회고를 기록한다.

- 푸시 알림: 매일 사용자가 지정한 시간에 체크인 알림 발송
- 당일 사용 데이터 세션 단위 전처리 후 AI 호출
- AI 자연어 기반 문장 생성 후 5개의 특이 사용 패턴 후보 제시
- K(Keep) / P(Problem) / T(Try) 패턴 선택 및 기록



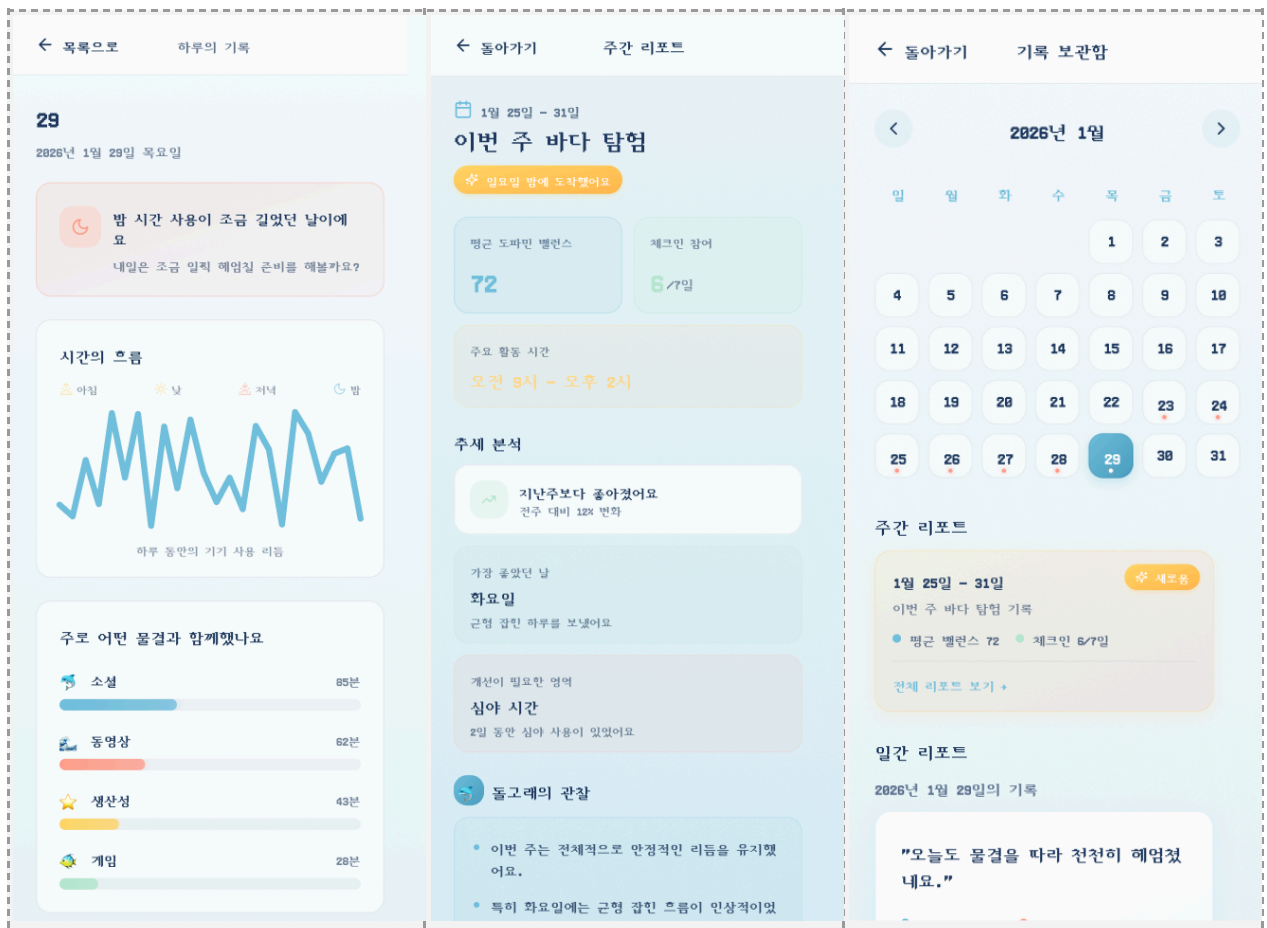
2. AI 개인화 리포트

2.1 데일리 리포트

- 사용자가 선택한 KPT 패턴 / 사용 흐름 / 주 사용 카테고리 정보 제공
- 당일 KPT 패턴과 온보딩 정보를 기반으로 AI 코멘트 생성

2.2 주간 리포트

- 데일리 리포트와 온보딩 정보를 기반으로 생성
- 점수 / 추세분석 / 체크인 참여 횟수 정보 제공
- AI 기반 주간 코멘트 생성
- 데일리/주간 리포트 모두 캘린더에서 날짜별 조회 가능



3. 소셜 기능

Google OAuth 기반 계정 연동을 바탕으로 소셜 기능을 제공한다.

- 친구 추가, 삭제, 신청 등 기본 친구 커넥션 기능 (서로 응원 알림 주고받기 — 랜덤 발송)
- 친구 사용자와 일주일 단위 그룹 챌린지 기능 (체크인 기록, 리포트 조회, 주간 리포트 생성 성공 등)
- 그룹 챌린지 달성 보상으로 골드/경험치를 난이도 및 지속 일수에 따라 차등 지급
- 그룹 상태 화면에서 다른 사용자들의 메인 캐릭터 확인 (같은 바다에서 유명하는 UI로 연대감 제공)

4. 게이미피케이션 요소

- DolphinPod만의 스토리와 세계관이 담긴 게임 엔진
- 일정 기준치 달성 시 재밌는 타이틀이 담긴 업적 달성 배지 부여
- 체크인 및 그룹 챌린지 보상으로 코인 획득 → 캐릭터·액세서리·배경·이펙트 커스텀 가능



기타 : Android 기기 사용 데이터 수집 구현 결과

수집되는 데이터 항목:

- 기기 잠금 해제 횟수
- 사용 권한 허용 상태
- 특정 앱 방문 횟수
- 앱별 총 사용 시간
- 특정 앱 최장 연속 사용 시간

구현 내용:

- 사용자의 기기 사용 데이터(앱별) 수집 및 저장
- FE-BE-DB 파이프라인 연동 (실시간 조회 가능)
- PostgreSQL 기반 Supabase DB 구축